

KTH Royal Institute of Technology

Omogen Heap

Simon Lindholm, Johan Sannemo, Mårten Wiman

Personalized layout preview

2026-10-20

1 Contest 1

2 Mathematics 1

3 Data structures 2

4 Numerical 5

5 Number theory 8

6 Combinatorial 10

7 Graph 11

8 Geometry 15

9 Strings 20

10 Various 22

11 Reference 24

Contest (1)

template.cpp

Usage:

```
int n;cin>>n; // read input, e.g.  5
cout<<n*n; // output 25
```

11 lines

```
#include<bits/stdc++.h>
using namespace std;
#define rep(i, a, b) for(int i=a; i<(b);++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
using i64=long long;
#define ll i64
#define vec vector
typedef pair<int, int>pii; typedef vector<int>vi;
int main() { cin.tie(0)->sync_with_stdio(0);
  cin.exceptions(cin.failbit); }
```

.bashrc

Usage:

```
source ~/.bashrc
# Run the aliases/functions shown below in that shell.
```

3 lines

```
alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++17 \
  -fsanitize=undefined,address'
xmodmap -e 'clear lock' -e 'keycode 66=less greater' #caps = <>
```

.vimrc

Usage:

```
# In Vim, source the file with :source ~/.vimrc
# Use the mappings shown below while editing a C++ file.
```

6 lines

```
set cin aw ai is ts=4 sw=4 tm=50 nu noeb bg=dark ru cul
sy on   |   im jk <esc>   |   im kj <esc>   |   no ; :
" Select region and then type :Hash to hash your selection.
" Useful for verifying that there aren't mistypes.
ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:space:]' \
 \| md5sum \| cut -c-6
```

hash.sh

Usage: `sh content/contest/hash.sh < solution.cpp`

3 lines

```
# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum |cut -c-6
```

troubleshoot.txt

Usage:

```
# Compare expected/actual output on a minimal failing case.
# Check the listed overflow, bounds and initialization cases.
```

52 lines

```
Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it.

Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?
```

Mathematics (2)

2.1 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

$$\begin{matrix} ax + by = e \\ cx + dy = f \end{matrix} \Rightarrow \begin{matrix} x = \frac{ed - bf}{ad - bc} \\ y = \frac{af - ec}{ad - bc} \end{matrix}$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A_i'}{\det A}$$

where A_i' is A with the i'th column replaced by b.

2.2 Recurrences

If $a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k}$, and $r_1, \ldots, r_k$ are distinct roots of $x^k - c_1 x^{k-1} - \cdots - c_k$, there are $d_1, \ldots, d_k$ s.t.

$$a_n = d_1 r_1^n + \cdots + d_k r_k^n.$$

Non-distinct roots r become polynomial factors, e.g. $a_n = (d_1 n + d_2) r^n$.

2.3 Trigonometry

$$\sin(v + w) = \sin v \cos w + \cos v \sin w$$
$$\cos(v + w) = \cos v \cos w - \sin v \sin w$$

$$\tan(v + w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$
$$\sin v + \sin w = 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2}$$
$$\cos v + \cos w = 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w.

$$a \cos x + b \sin x = r \cos(x - \phi)$$
$$a \sin x + b \cos x = r \sin(x + \phi)$$

where $r = \sqrt{a^2 + b^2}, \phi = \text{atan2}(b, a)$.

2.4 Geometry

2.4.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \frac{a+b+c}{2}$

Area: $A = \sqrt{p(p-a)(p-b)(p-c)}$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two): $s_a = \sqrt{bc\left[1 - \left(\frac{a}{b+c}\right)^2\right]}$

Law of sines: $\frac{\sin\alpha}{a} = \frac{\sin\beta}{b} = \frac{\sin\gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc\cos\alpha$

Law of tangents: $\frac{a+b}{a-b} = \frac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

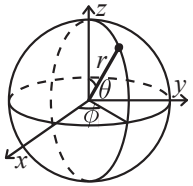
2.4.2 Quadrilaterals

With side lengths a, b, c, d, diagonals e, f, diagonals angle θ, area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin\theta = F\tan\theta = \sqrt{4e^2f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is $180°$, $ef = ac + bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

2.4.3 Spherical coordinates



$$x = r\sin\theta\cos\phi \qquad r = \sqrt{x^2+y^2+z^2}$$
$$y = r\sin\theta\sin\phi \qquad \theta = \text{acos}(z/\sqrt{x^2+y^2+z^2})$$
$$z = r\cos\theta \qquad \phi = \text{atan2}(y, x)$$

2.5 Derivatives/Integrals

$$\frac{d}{dx}\arcsin x = \frac{1}{\sqrt{1-x^2}} \qquad \frac{d}{dx}\arccos x = -\frac{1}{\sqrt{1-x^2}}$$
$$\frac{d}{dx}\tan x = 1 + \tan^2 x \qquad \frac{d}{dx}\arctan x = \frac{1}{1+x^2}$$
$$\int \tan ax = -\frac{\ln|\cos ax|}{a} \qquad \int x\sin ax = \frac{\sin ax - ax\cos ax}{a^2}$$
$$\int e^{-x^2} = \frac{\sqrt{\pi}}{2}\text{erf}(x) \qquad \int xe^{ax}dx = \frac{e^{ax}}{a^2}(ax-1)$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.6 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c-1}, c \neq 1$$

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$
$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n+1)(n+1)}{6}$$
$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$
$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.7 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots, (-\infty < x < \infty)$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \leq 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, (-\infty < x < \infty)$$

2.8 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x. It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x xp_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

2.8.1 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $\text{U}(a, b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2},\ \sigma^2 = \frac{(b-a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $\text{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda},\ \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

Data structures (3)

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. To get a map, change `null_type`. For duplicates, use (value, unique ID) keys; the example uses IDs 0 and 1.

Usage:

```
Tree<int>t;t.insert(8);t.insert(10);
cout<<t.order_of_key(10); // 1
cout<<*t.find_by_order(0); // 8
Tree<pii>ms;ms.insert({8, 0});ms.insert({8, 1});
cout<<ms.order_of_key({8, INT_MIN}); // 0, strictly less
cout<<ms.find_by_order(1)->first; // 8, second copy
```

Time: $\mathcal{O}(\log N)$

11 lines

```
#include<bits/extc++.h>
using namespace __gnu_pbds; template<class T>
using Tree=tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;
void example() { Tree<int>t, t2; t.insert(8);
  auto it=t.insert(10).first; assert(it==t.lower_bound(9));
  assert(t.order_of_key(10)==1);
  assert(t.order_of_key(11)==2);
  assert(*t.find_by_order(0)==8);
  t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

HashMap.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

Usage:

```
h[7]=3;auto it=h.find(7);
if(it!=h.end())cout<<it->second; // 3
```

7 lines

```
#include<bits/extc++.h>
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
  const uint64_t C=ll(4e18*acos(0))|71;
  ll operator()(ll x) const { return __builtin_bswap64(x*C); }
};
__gnu_pbds::gp_hash_table<ll, int, chash>h({}, {}, {}, {}, {
    1<<16});
```

LazySegmentTree.h

Description: Generic lazy segment tree using Info, Tag and apply. Zero-based [l,r). kth(k) returns the zero-based index of the 1-based rank, or -1; requires a nonempty tree and nonnegative frequency sums.

Usage:

```
vec<Info>a={Info(1), Info(5), Info(3)};
LazySegmentTree<Info, Tag>st(a);
st.modify(0, 2, Tag{2}); // add 2 to [0,2)
auto ans=st.query(0, 3);
cout<<ans.sum<<' '<<ans.max_val; // 13 7
vec<Info>f={Info(1), Info(0), Info(1)};
LazySegmentTree<Info, Tag>freq(f);
cout<<freq.kth(2); // index:  2 (rank starts at 1)
```

Time: $\mathcal{O}(N)$ build and space; $\mathcal{O}(\log N)$ per operation.

67 lines

```
struct Info { ll sum=0;ll max_val=-1e18;ll len=0;
 Info() {}
 Info(ll val):sum(val), max_val(val), len(1) {}
 friend Info operator+(const Info&a, const Info&b) {
  Info c;c.sum=a.sum+b.sum;
  c.max_val=max(a.max_val, b.max_val);
  c.len=a.len+b.len;return c; }
};
struct Tag {
 ll add=0;bool has_tag() const {return add!=0;}
 void clear() {add=0;}
};
void apply(Info&info, const Tag&tag) {
 if (!tag.has_tag()) return;
 info.sum+=tag.add*info.len;info.max_val+=tag.add; }
void apply(Tag&old_tag, const Tag&new_tag) {
 old_tag.add+=new_tag.add; }
template<class Info, class Tag>
class LazySegmentTree {
private:
 int n;vec<Info>info;vec<Tag>tag;
 void push_up(int p) {info[p]=info[2*p]+info[2*p+1];}
 void apply_node(int p, const Tag&v) {
  apply(info[p], v);apply(tag[p], v); }
 void push_down(int p) {
  if (tag[p].has_tag()) {
   apply_node(2*p, tag[p]);apply_node(2*p+1, tag[p]);
   tag[p].clear(); }
 }
 void build(int p, int l, int r, const vec<Info>&init_info) {
  if (l+1==r) {info[p]=init_info[l];return;}
  int m=(l+r)/2;build(2*p, l, m, init_info);
  build(2*p+1, m, r, init_info);push_up(p); }
 void modify(int p, int l, int r, int ql, int qr, const Tag&v)
     {
  if (ql<=l&&r<=qr) {apply_node(p, v);return;}
  push_down(p);int m=(l+r)/2;
  if (ql<m) modify(2*p, l, m, ql, qr, v);
  if (qr>m) modify(2*p+1, m, r, ql, qr, v);
  push_up(p); }
 Info query(int p, int l, int r, int ql, int qr) {
  if (ql<=l&&r<=qr) {return info[p];}
  push_down(p);int m=(l+r)/2;
  if (qr<=m) return query(2*p, l, m, ql, qr);
  if (ql>=m) return query(2*p+1, m, r, ql, qr);
  return query(2*p, l, m, ql, qr)+query(2*p+1, m, r, ql, qr);
 }
 int kth(int p, int l, int r, ll k) {
  if (l+1==r) {return l;}
  push_down(p);int m=(l+r)/2; ll left_count=info[2*p].sum;
  if (k<=left_count) { return kth(2*p, l, m, k);
  } else { return kth(2*p+1, m, r, k-left_count);
  }
 }
public:
 LazySegmentTree(const vec<Info>&init_info) {
  n=init_info.size();info.assign(4*n, Info());
  tag.assign(4*n, Tag());
  if (n>0) build(1, 0, n, init_info); }
 void modify(int l, int r, const Tag&v) {
  if (l<r) modify(1, 0, n, l, r, v); }
 Info query(int l, int r) {
  if (l<r) return query(1, 0, n, l, r);
  return Info(); }
 int kth(ll k) {
  if (k<1||k>info[1].sum) return-1;
  return kth(1, 0, n, k); }
};
```

XorBasis.h

Description: 30-bit linear-basis fragment: reads n nonnegative integers from cin and prints their maximum subset XOR. Paste inside a function after declaring n.

Usage:

```
int n=3; // cin:  1 2 3
// Paste fragment here.
// output:  3
```

Time: $\mathcal{O}(30N)$ time, $\mathcal{O}(30)$ space.

11 lines

```
bool has_zero=false; // Nonempty subset with XOR 0.
vec<ll>basis(30, 0); for (int i=0; i<n; i++) { ll x;
  cin>>x; for (int j=29; j>=0; j--) { if ((x>>j)&1) {
      if (!basis[j]) { basis[j]=x; break; }
      x^=basis[j]; }
  }
  if (!x) has_zero=true; }
ll max_xor=0; for (int j=29; j>=0; j--) {
  if ((max_xor^basis[j])>max_xor) { max_xor^=basis[j]; }
}
cout<<max_xor<<"\\n";
```

PersistentSegmentTree.h

Description: Persistent segment tree. Note MXN is O(2n + qlog(n)). Queries are [l,r].

Usage:

```
PersistentSegtree st(3, 100);
vec<int>a={1, 2, 3};int r0=st.build(a);
int r1=st.set(r0, 1, 9);
cout<<st.rangeSum(r0, 0, 2); // 6, old version
cout<<st.rangeSum(r1, 0, 2); // 13, new version
```

Time: $\mathcal{O}(\log N)$

35 lines

```
struct PersistentSegtree {
  struct Node { ll sum=0; int l=0, r=0; };
  const int n; vector<Node>tree; int timer=1;
  Node join(int l, int r) { return Node{tree[l].sum+tree[r].sum
      , l, r}; }
  int build(int tl, int tr, const vector<int>&arr) {
    if (tl==tr) {
      tree[timer]={arr[tl], 0, 0};
      return timer++; }
    int mid=(tl+tr)/2;
    tree[timer]=join(build(tl, mid, arr), build(mid+1, tr, arr)
        );
    return timer++; }
  int set(int v, int pos, int val, int tl, int tr) {
    if (tl==tr) {
      tree[timer]={val, 0, 0};
      return timer++; }
    int mid=(tl+tr)/2;
    if (pos<=mid) {
      tree[timer]=join(set(tree[v].l, pos, val, tl, mid), tree[
          v].r);
    } else {
      tree[timer]=join(tree[v].l, set(tree[v].r, pos, val, mid
          +1, tr));
    }
    return timer++; }
  ll rangeSum(int v, int ql, int qr, int tl, int tr) {
    if (qr<tl||tr<ql) { return 0ll; }
    if (ql<=tl&&tr<=qr) { return tree[v].sum; }
    int mid=(tl+tr)/2;
    return rangeSum(tree[v].l, ql, qr, tl, mid)+
           rangeSum(tree[v].r, ql, qr, mid+1, tr); }
  PersistentSegtree(int n, int MXN):n(n), tree(MXN) {}
  int build(const vector<int>&arr) { return build(0, n-1, arr);
      }
  int set(int root, int pos, int val) { return set(root, pos,
      val, 0, n-1); }
  ll rangeSum(int root, int l, int r) { return rangeSum(root, l
      , r, 0, n-1); }
  int addCopy(int root) { tree[timer]=tree[root];
    return timer++; }
};
```

UnionFind.h

Description: Disjoint-set data structure.

Usage:

```
UF uf(4);uf.join(0, 1);
cout<<uf.sameSet(0, 1)<<' '<<uf.size(0); // 1 2
```

Time: $\mathcal{O}(\alpha(N))$

10 lines
```
struct UF { vi e;
  UF(int n):e(n, -1) {}
  bool sameSet(int a, int b) { return find(a)==find(b); }
  int size(int x) { return-e[find(x)]; }
  int find(int x) { return e[x]<0?x:e[x]=find(e[x]); }
  bool join(int a, int b) { a=find(a), b=find(b);
    if (a==b) return false;
    if (e[a]>e[b]) swap(a, b);
    e[a]+=e[b]; e[b]=a; return true; }
};
```

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip st, time() and rollback().

Usage:
```
RollbackUF uf(3);int t=uf.time();uf.join(0, 1);
cout<<uf.size(0); // 2
uf.rollback(t);cout<<uf.size(0); // 1
```

Time: $\mathcal{O}(\log(N))$

15 lines
```
struct RollbackUF { vi e; vector<pii>st;
  RollbackUF(int n):e(n, -1) {}
  int size(int x) { return-e[find(x)]; }
  int find(int x) { return e[x]<0?x:find(e[x]); }
  int time() { return sz(st); }
  void rollback(int t) {
    for (int i=time(); i-->t;)
      e[st[i].first]=st[i].second; st.resize(t); }
  bool join(int a, int b) { a=find(a), b=find(b);
    if (a==b) return false;
    if (e[a]>e[b]) swap(a, b);
    st.push_back({a, e[a]});
    st.push_back({b, e[b]}); e[a]+=e[b]; e[b]=a; return true;
  }
};
```

SubMatrix.h

Description: Calculate submatrix sums quickly, given upper-left and lower-right corners (half-open).

Usage:
```
vec<vec<int>>a={{1, 2}, {3, 4}};
SubMatrix<int>sm(a);
cout<<sm.sum(0, 0, 2, 2); // 10
```

Time: $\mathcal{O}\left(N^2 + Q\right)$

8 lines
```
template<class T>
struct SubMatrix { vector<vector<T>>p;
  SubMatrix(vector<vector<T>>&v) { int R=sz(v), C=sz(v[0]);
    p.assign(R+1, vector<T>(C+1)); rep(r, 0, R) rep(c, 0, C)
      p[r+1][c+1]=v[r][c]+p[r][c+1]+p[r+1][c]-p[r][c]; }
  T sum(int u, int l, int d, int r) {
    return p[d][r]-p[d][l]-p[u][r]+p[u][l]; }
};
```

Matrix.h

Description: Basic operations on square matrices.

Usage:
```
Matrix<ll, 2>A;
A.d={{{1, 1}, {1, 0}}};array<ll, 2>x={1, 0};
auto y=(A^5)*x;
cout<<y[0]<<' '<<y[1]; // 8 5
```

16 lines
```
template<class T, int N>struct Matrix { typedef Matrix M;
  array<array<T, N>, N>d{};
  M operator*(const M&m) const { M a;
    rep(i, 0, N) rep(j, 0, N)
      rep(k, 0, N) a.d[i][k]+=d[i][j]*m.d[j][k]; return a; }
  array<T, N>operator*(const array<T, N>&vec) const {
    array<T, N>ret{};
    rep(i, 0, N) rep(j, 0, N) ret[i]+=d[i][j]*vec[j];
    return ret; }
  M operator^(ll p) const { assert(p>=0); M a, b(*this);
    rep(i, 0, N) a.d[i][i]=1;
    while (p) {
      if (p&1) a=a*b;
      b=b*b; p>>=1; }
    return a; }
};
```

LineContainer.h

Description: Container where you can add lines of the form kx+m, and query maximum values at points x. Useful for dynamic programming ("convex hull trick").

Usage:
```
LineContainer lc;lc.add(2, 1);lc.add(-1, 4);
cout<<lc.query(3); // 7
```

Time: $\mathcal{O}(\log N)$

23 lines
```
struct Line { mutable ll k, m, p;
  bool operator<(const Line&o) const { return k<o.k; }
  bool operator<(ll x) const { return p<x; }
};
struct LineContainer:multiset<Line, less<>>{
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const ll inf=LLONG_MAX;
  ll div(ll a, ll b) { // floored division
    return a/b-((a^b)<0&&a%b); }
  bool isect(iterator x, iterator y) {
    if (y==end()) return x->p=inf, 0;
    if (x->k==y->k) x->p=x->m>y->m?inf:-inf;
    else x->p=div(y->m-x->m, x->k-y->k);
    return x->p>=y->p; }
  void add(ll k, ll m) {
    auto z=insert({k, m, 0}), y=z++, x=y;
    while (isect(y, z)) z=erase(z);
    if (x!=begin()&&isect(--x, y)) isect(x, y=erase(y));
    while ((y=x)!=begin()&&(--x)->p>=y->p)
      isect(x, erase(y)); }
  ll query(ll x) { assert(!empty()); auto l=*lower_bound(x);
    return l.k*x+l.m; }
};
```

Treap.h

Description: A short self-balancing tree. It acts as a sequential container with log-time splits/joins, and is easy to augment with additional data.

Usage:
```
Node*root=nullptr;
root=ins(root, new Node(4), 0);
root=ins(root, new Node(7), 1);
auto [a, b]=split(root, 1);
cout<<a->val<<' '<<b->val; // 4 7
root=merge(a, b);
```

Time: $\mathcal{O}(\log N)$

34 lines
```
struct Node { Node*l=0, *r=0; int val, y, c=1;
  Node(int val):val(val), y(rand()) {}
  void recalc(); };
int cnt(Node*n) { return n?n->c:0; }
void Node::recalc() { c=cnt(l)+cnt(r)+1; }
template<class F>void each(Node*n, F f) {
  if (n) { each(n->l, f); f(n->val); each(n->r, f); }
}
pair<Node*, Node*>split(Node*n, int k) {
  if (!n) return {};
  if (cnt(n->l)>=k) { // "n->val >= k" for lower_bound(k)
    auto [L, R]=split(n->l, k); n->l=R; n->recalc();
    return {L, n};
  } else {
    auto [L, R]=split(n->r, k-cnt(n->l)-1); // and just "k"
    n->r=L; n->recalc();
    return {n, R};
  }
}
Node*merge(Node*l, Node*r) {
  if (!l) return r;
  if (!r) return l;
  if (l->y>r->y) { l->r=merge(l->r, r);
    return l->recalc(), l;
  } else { r->l=merge(l, r->l);
    return r->recalc(), r; }
}
Node*ins(Node*t, Node*n, int pos) { auto [l, r]=split(t, pos);
  return merge(merge(l, n), r); }
// Example application: move the range [l, r) to index k
void move(Node*&t, int l, int r, int k) { Node*a, *b, *c;
  tie(a, b)=split(t, l); tie(b, c)=split(b, r-l);
  if (k<=l) t=merge(ins(a, b, k), c);
  else t=merge(a, ins(c, b, k-r)); }
```

FenwickTree.h

Description: Computes partial sums a[0] + a[1] + ... + a[pos - 1], and updates single elements a[i], taking the difference between the old and new value.

Usage:
```
FT ft(5);ft.update(2, 7);
cout<<ft.query(3); // 7, sum on [0,3)
FT diff(5);diff.update(1, 4);diff.update(4, -4);
cout<<diff.query(3); // 4, value at index 2 after [1,4)+=4
```

Time: Both operations are $\mathcal{O}(\log N)$.

17 lines
```
struct FT { vector<ll>s;
  FT(int n):s(n) {}
  void update(int pos, ll dif) { // a[pos] += dif
    for (; pos<sz(s); pos|=pos+1) s[pos]+=dif; }
  ll query(int pos) { // sum of values in [0, pos)
    ll res=0;
    for (; pos>0; pos&=pos-1) res+=s[pos-1];
    return res; }
  int lower_bound(ll sum) {// min pos st sum of [0, pos] >= sum
    // Returns n if no sum is >= sum, or -1 if empty sum is.
    if (sum<=0) return-1;
    int pos=0;
    for (int pw=1<<25; pw; pw>>=1) {
      if (pos+pw<=sz(s)&&s[pos+pw-1]<sum)
        pos+=pw, sum-=s[pos-1]; }
    return pos; }
};
```

FenwickTree2d.h

Description: Computes sums a[i,j] for all i<I, j<J, and increases single elements a[i,j]. Requires that the elements to be updated are known in advance (call fakeUpdate() before init()).

Usage:
```
FT2 ft(4);ft.fakeUpdate(1, 2);ft.init();
ft.update(1, 2, 7);cout<<ft.query(2, 3); // 7
```

Time: $\mathcal{O}\left(\log^2 N\right)$. (Use persistent segment trees for $\mathcal{O}(\log N)$.)

"FenwickTree.h"

15 lines
```
struct FT2 { vector<vi>ys; vector<FT>ft;
  FT2(int limx):ys(limx) {}
  void fakeUpdate(int x, int y) {
    for (; x<sz(ys); x|=x+1) ys[x].push_back(y); }
  void init() {
    for (vi&v:ys) sort(all(v)), ft.emplace_back(sz(v)); }
  int ind(int x, int y) {
    return (int)(lower_bound(all(ys[x]), y)-ys[x].begin()); }
  void update(int x, int y, ll dif) {
```

```
    for (; x<sz(ys); x|=x+1)
      ft[x].update(ind(x, y), dif); }
  ll query(int x, int y) { ll sum=0;
    for (; x; x&=x-1)
      sum+=ft[x-1].query(ind(x-1, y)); return sum; }
};
```

RMQ.h

Description: Range Minimum Queries on an array. Returns min(V[a], V[a + 1], ... V[b - 1]) in constant time.
Usage:
`vec<int>a={4, 2, 7};RMQ<int>rmq(a);`
`cout<<rmq.query(0, 3); // 2`
Time: $\mathcal{O}(|V|\log|V|+Q)$

12 lines

```
template<class T>
struct RMQ { vector<vector<T>>jmp;
  RMQ(const vector<T>&V):jmp(1, V) {
    for (int pw=1, k=1; pw*2<=sz(V); pw*=2, ++k) {
      jmp.emplace_back(sz(V)-pw*2+1); rep(j, 0, sz(jmp[k]))
        jmp[k][j]=min(jmp[k-1][j], jmp[k-1][j+pw]); }
  }
  T query(int a, int b) {
    assert(a<b); // or return inf if a == b
    int dep=31-__builtin_clz(b-a);
    return min(jmp[dep][a], jmp[dep][b-(1<<dep)]); }
};
```

MoQueries.h

Description: Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change `step` to add/remove the edge (a, c) and remove the initial `add` call (but keep `in`). For range sums, let `add` increase `cur` by `a[ind]`, `del` decrease it, and `calc` return `cur`.
Usage:
`auto ans=mo({{0, 2}, {1, 3}});`
`cout<<ans[0]<<' '<<ans[1]; // 5 8 for a={2,3,5}`
Time: $\mathcal{O}\left(N\sqrt{Q}\right)$

39 lines

```
void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... } // remove a[ind]
int calc() { ... } // compute current answer
vi mo(vector<pii>Q) { int L=0, R=0, blk=350; // ~N/sqrt(Q)
  vi s(sz(Q)), res=s;
#define K(x) pii(x.first/blk, x.second^-(x.first/blk&1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t){ return K(Q[s])<K(Q[t]); });
  for (int qi:s) { pii q=Q[qi];
    while (L>q.first) add(--L, 0);
    while (R<q.second) add(R++, 1);
    while (L<q.first) del(L++, 0);
    while (R>q.second) del(--R, 1);
    res[qi]=calc(); }
  return res; }
vi moTree(vector<array<int, 2>>Q, vector<vi>&ed, int root=0){
  int N=sz(ed), pos[2]={}, blk=350; // ~N/sqrt(Q)
  vi s(sz(Q)), res=s, I(N), L(N), R(N), in(N), par(N);
  add(0, 0), in[0]=1;
  auto dfs=[&](int x, int p, int dep, auto&f)->void {
    par[x]=p; L[x]=N;
    if (dep) I[x]=N++;
    for (int y:ed[x]) if (y!=p) f(y, x, !dep, f);
    if (!dep) I[x]=N++;
    R[x]=N; };
  dfs(root, -1, 0, dfs);
#define K(x) pii(I[x[0]]/blk, I[x[1]]^-(I[x[0]]/blk&1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t){ return K(Q[s])<K(Q[t]); });
  for (int qi:s) rep(end, 0, 2) {
    int&a=pos[end], b=Q[qi][end], i=0;
#define step(c) { if (in[c]) { del(a, end); in[a]=0; } \
                  else { add(c, end); in[c]=1; } a=c; }
    while (!(L[b]<=L[a]&&R[a]<=R[b]))
      I[i++]=b, b=par[b];
    while (a!=b) step(par[a]);
    while (i--) step(I[i]);
    if (end) res[qi]=calc(); }
  return res; }
```

Numerical (4)

4.1 Polynomials and recurrences

Polynomial.h

Usage:
`Poly p{{2, -3, 1}};cout<<p(3); // 2`
`p.diff();cout<<p(3); // 3 (derivative 2*x-3)`

10 lines

```
struct Poly { vector<double>a;
  double operator()(double x) const { double val=0;
    for (int i=sz(a); i--;) (val*=x)+=a[i];
    return val; }
  void diff() { rep(i, 1, sz(a)) a[i-1]=i*a[i]; a.pop_back();
  }
  void divroot(double x0) { double b=a.back(), c; a.back()=0;
    for(int i=sz(a)-1; i--;) c=a[i], a[i]=a[i+1]*x0+b, b=c;
    a.pop_back(); }
};
```

PolyRoots.h

Description: Finds the real roots to a polynomial.
Usage:
`auto roots=polyRoots(Poly{{2, -3, 1}}, 0, 3);`
`for(double x:roots)cout<<x<<' '; // approximately 1 2`
Time: $\mathcal{O}\left(n^2\log(1/\epsilon)\right)$

"Polynomial.h" 15 lines

```
vector<double>polyRoots(Poly p, double xmin, double xmax) {
  if (sz(p.a)==2) { return {-p.a[0]/p.a[1]}; }
  vector<double>ret; Poly der=p; der.diff();
  auto dr=polyRoots(der, xmin, xmax); dr.push_back(xmin-1);
  dr.push_back(xmax+1); sort(all(dr));
  rep(i, 0, sz(dr)-1) { double l=dr[i], h=dr[i+1];
    bool sign=p(l)>0;
    if (sign^(p(h)>0)) {
      rep(it, 0, 60) { // while (h - l > 1e-8)
        double m=(l+h)/2, f=p(m);
        if ((f<=0)^sign) l=m;
        else h=m; }
      ret.push_back((l+h)/2); }
  }
  return ret; }
```

PolyInterpolate.h

Description: Given n points (x[i], y[i]), computes an n-1-degree polynomial p that passes through them: $p(x) = a[0] * x^0 + ... + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \dots n-1$.
Usage:
`auto a=interpolate({0, 1, 2}, {1, 2, 5}, 3);`
`cout<<a[0]<<' '<<a[2]; // approximately 1 1`
Time: $\mathcal{O}\left(n^2\right)$

7 lines

```
typedef vector<double>vd;
vd interpolate(vd x, vd y, int n) { vd res(n), temp(n);
  rep(k, 0, n-1) rep(i, k+1, n)
    y[i]=(y[i]-y[k])/(x[i]-x[k]); double last=0; temp[0]=1;
  rep(k, 0, n) rep(i, 0, n) { res[i]+=y[k]*temp[i];
    swap(last, temp[i]); temp[i]-=last*x[k]; }
  return res; }
```

BerlekampMassey.h

Description: Recovers any n-order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.
Usage:
`auto c=berlekampMassey({0, 1, 1, 3, 5, 11});`
`cout<<c[0]<<' '<<c[1]; // 1 2`
Time: $\mathcal{O}\left(N^2\right)$

"../number-theory/ModPow.h" 13 lines

```
vector<ll>berlekampMassey(vector<ll>s) {
  int n=sz(s), L=0, m=0; vector<ll>C(n), B(n), T; C[0]=B[0]=1;
  ll b=1;
  rep(i, 0, n) {++m; ll d=s[i]%mod;
    rep(j, 1, L+1) d=(d+C[j]*s[i-j])%mod;
    if (!d) continue;
    T=C; ll coef=d*modpow(b, mod-2)%mod;
    rep(j, m, n) C[j]=(C[j]-coef*B[j-m])%mod;
    if (2*L>i) continue;
    L=i+1-L; B=T; b=d; m=0; }
  C.resize(L+1); C.erase(C.begin());
  for (ll&x:C) x=(mod-x)%mod;
  return C; }
```

LinearRecurrence.h

Description: Generates the k'th term of an n-order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \dots \geq n-1]$ and $tr[0 \dots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey. Define `mod` before use; the example uses `mod=5`.
Usage: `cout<<linearRec({0, 1}, {1, 1}, 8); // 21 % mod`
Time: $\mathcal{O}\left(n^2\log k\right)$

14 lines

```
typedef vector<ll>Poly;
ll linearRec(Poly S, Poly tr, ll k) { int n=sz(tr);
  auto combine=[&](Poly a, Poly b) { Poly res(n*2+1);
    rep(i, 0, n+1) rep(j, 0, n+1)
      res[i+j]=(res[i+j]+a[i]*b[j])%mod;
    for (int i=2*n; i>n;--i) rep(j, 0, n)
      res[i-1-j]=(res[i-1-j]+res[i]*tr[j])%mod;
    res.resize(n+1); return res; };
  Poly pol(n+1), e(pol); pol[0]=e[1]=1;
  for (++k; k; k/=2) {
    if (k%2) pol=combine(pol, e);
    e=combine(e, e); }
  ll res=0; rep(i, 0, n) res=(res+pol[i+1]*S[i])%mod;
  return res; }
```

4.2 Optimization

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum and no local maximum. The maximum error in the result is eps. Works equally well for maximization with a small change in the code. See TernarySearch.h in the Various chapter for a discrete version.
Usage:
`auto f=+[](double x){return (x-2)*(x-2);};`
`double x=gss(-5, 5, f);cout<<x<<' '<<f(x); // about 2 0`
Time: $\mathcal{O}\left(\log((b-a)/\epsilon)\right)$

10 lines

```
double gss(double a, double b, double (*f)(double)) {
  double r=(sqrt(5)-1)/2, eps=1e-7;
  double x1=b-r*(b-a), x2=a+r*(b-a);
  double f1=f(x1), f2=f(x2);
```

```
  while (b-a>eps)
    if (f1<f2) { //change to > to find maximum
      b=x2; x2=x1; f2=f1; x1=b-r*(b-a); f1=f(x1);
    } else { a=x1; x1=x2; f1=f2;
      x2=a+r*(b-a); f2=f(x2); }
  return a; }
```

HillClimbing.h

Description: Poor man's optimization for unimodal functions.
Usage:

```
auto [value, p]=hillClimb(P{0, 0}, [](P p){
return (p[0]-2)*(p[0]-2)+p[1]*p[1];});
cout<<value<<' '<<p[0]; // about 0 2
```

9 lines

```
typedef array<double, 2>P;
template<class F>pair<double, P>hillClimb(P start, F f) {
  pair<double, P>cur(f(start), start);
  for (double jmp=1e9; jmp>1e-20; jmp/=2) {
    rep(j, 0, 100) rep(dx, -1, 2) rep(dy, -1, 2) {
      P p=cur.second; p[0]+=dx*jmp; p[1]+=dy*jmp;
      cur=min(cur, make_pair(f(p), p)); }
  }
  return cur; }
```

Integrate.h

Description: Simple integration of a function over an interval using Simpson's rule. The error should be proportional to h^4, although in practice you will want to verify that the result is stable to desired precision when epsilon changes.
Usage:

```
double area=quad(0, 1, [](double x){return x*x;});
cout<<area; // about 1/3
```

4 lines

```
template<class F>
double quad(double a, double b, F f, const int n=1000) {
  double h=(b-a)/2/n, v=f(a)+f(b); rep(i, 1, n*2)
    v+=f(a+i*h)*(i&1?4:2); return v*h/3; }
```

IntegrateAdaptive.h

Description: Fast integration using an adaptive Simpson's rule.
Usage:

```
double area=quad(0, 1, [](double x){return x*x;}, 1e-9);
cout<<area; // about 1/3
```

11 lines

```
typedef double d;
#define S(a, b) (f(a)+4*f((a+b)/2)+f(b))*(b-a)/6
template<class F>
d rec(F&f, d a, d b, d eps, d S) { d c=(a+b)/2;
  d S1=S(a, c), S2=S(c, b), T=S1+S2;
  if (abs(T-S)<=15*eps||b-a<1e-10)
    return T+(T-S)/15;
  return rec(f, a, c, eps/2, S1)+rec(f, c, b, eps/2, S2); }
template<class F>
d quad(d a, d b, F f, d eps=1e-8) {
  return rec(f, a, b, eps, S(a, b)); }
```

Simplex.h

Description: Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \leq b$, $x \geq 0$. Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.
Usage:

```
vvd A={{1, 1}, {1, 0}, {0, 1}};
vd b={4, 2, 3}, c={3, 2}, x;
LPSolver lp(A, b, c);T value=lp.solve(x);
cout<<value<<' '<<x[0]<<' '<<x[1]; // 10 2 2
```

Time: $\mathcal{O}(NM * \#pivots)$, where a pivot may be e.g. an edge relaxation. $\mathcal{O}(2^n)$ in the general case.

42 lines

```
typedef double T; // long double, Rational, double + mod<P>...
typedef vector<T>vd; typedef vector<vd>vvd;
const T eps=1e-8, inf=1/.0;
#define MP make_pair
#define ltj(X) if(s==-1||MP(X[j], N[j])<MP(X[s], N[s])) s=j
struct LPSolver { int m, n; vi N, B; vvd D;
  LPSolver(const vvd&A, const vd&b, const vd&c):
    m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
      rep(i, 0, m) rep(j, 0, n) D[i][j]=A[i][j];
      rep(i, 0, m) { B[i]=n+i; D[i][n]=-1; D[i][n+1]=b[i];}
      rep(j, 0, n) { N[j]=j; D[m][j]=-c[j]; }
      N[n]=-1; D[m+1][n]=1; }
  void pivot(int r, int s) { T*a=D[r].data(), inv=1/a[s];
    rep(i, 0, m+2) if (i!=r&&abs(D[i][s])>eps) {
      T*b=D[i].data(), inv2=b[s]*inv;
      rep(j, 0, n+2) b[j]-=a[j]*inv2; b[s]=a[s]*inv2; }
    rep(j, 0, n+2) if (j!=s) D[r][j]*=inv;
    rep(i, 0, m+2) if (i!=r) D[i][s]*=-inv; D[r][s]=inv;
    swap(B[r], N[s]); }
  bool simplex(int phase) { int x=m+phase-1;
    for (;;) { int s=-1;
      rep(j, 0, n+1) if (N[j]!=-phase) ltj(D[x]);
      if (D[x][s]>=-eps) return true;
      int r=-1;
      rep(i, 0, m) {
        if (D[i][s]<=eps) continue;
        if (r==-1||MP(D[i][n+1]/D[i][s], B[i])
                  <MP(D[r][n+1]/D[r][s], B[r])) r=i; }
      if (r==-1) return false;
      pivot(r, s); }
  }
  T solve(vd&x) { int r=0;
    rep(i, 1, m) if (D[i][n+1]<D[r][n+1]) r=i;
    if (D[r][n+1]<-eps) { pivot(r, n);
      if (!simplex(2)||D[m+1][n+1]<-eps) return-inf;
      rep(i, 0, m) if (B[i]==-1) { int s=0;
        rep(j, 1, n+1) ltj(D[i]); pivot(i, s); }
    }
    bool ok=simplex(1); x=vd(n);
    rep(i, 0, m) if (B[i]<n) x[B[i]]=D[i][n+1];
    return ok?D[m][n+1]:inf; }
};
```

4.3 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.
Usage:

```
vec<vec<double>>a={{2, 1}, {3, 4}}, copy=a;
double value=det(copy);cout<<value; // 5; a unchanged
```

Time: $\mathcal{O}(N^3)$

11 lines

```
double det(vector<vector<double>>&a) {
  int n=sz(a); double res=1;
  rep(i, 0, n) { int b=i;
    rep(j, i+1, n) if (fabs(a[j][i])>fabs(a[b][i])) b=j;
    if (i!=b) swap(a[i], a[b]), res*=-1;
    res*=a[i][i];
    if (res==0) return 0;
    rep(j, i+1, n) { double v=a[j][i]/a[i][i];
      if (v!=0) rep(k, i+1, n) a[j][k]-=v*a[i][k]; }
  }
  return res; }
```

IntDeterminant.h

Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.
Usage:

```
vec<vec<ll>>a={{2, 1}, {3, 4}};
ll value=det(a);cout<<value; // 5; a is modified
```

Time: $\mathcal{O}(N^3)$

13 lines

```
const ll mod=12345;
ll det(vector<vector<ll>>&a) { int n=sz(a); ll ans=1;
  rep(i, 0, n) {
    rep(j, i+1, n) {
      while (a[j][i]!=0) { // gcd step
        ll t=a[i][i]/a[j][i];
        if (t) rep(k, i, n)
          a[i][k]=(a[i][k]-a[j][k]*t)%mod; swap(a[i], a[j]);
        ans*=-1; }
    }
    ans=ans*a[i][i]%mod;
    if (!ans) return 0; }
  return (ans+mod)%mod; }
```

SolveLinear.h

Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Usage:

```
vec<vd>A={{2, 1}, {1, -1}};vd b={5, 1}, x(2);
int rank=solveLinear(A, b, x);
if(rank!=-1)cout<<x[0]<<' '<<x[1]; // 2 1
```

Time: $\mathcal{O}(n^2 m)$

21 lines

```
typedef vector<double>vd; const double eps=1e-12;
int solveLinear(vector<vd>&A, vd&b, vd&x) {
  int n=sz(A), m=sz(x), rank=0, br, bc;
  if (n) assert(sz(A[0])==m);
  vi col(m); iota(all(col), 0);
  rep(i, 0, n) { double v, bv=0; rep(r, i, n) rep(c, i, m)
      if ((v=fabs(A[r][c]))>bv)
        br=r, bc=c, bv=v;
    if (bv<=eps) { rep(j, i, n) if (fabs(b[j])>eps) return-1;
      break; }
    swap(A[i], A[br]); swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j, 0, n) swap(A[j][i], A[j][bc]); bv=1/A[i][i];
    rep(j, i+1, n) { double fac=A[j][i]*bv; b[j]-=fac*b[i];
      rep(k, i+1, m) A[j][k]-=fac*A[i][k]; }
    rank++; }
  x.assign(m, 0);
  for (int i=rank; i--;) { b[i]/=A[i][i];
    x[col[i]]=b[i]; rep(j, 0, i) b[j]-=A[j][i]*b[i]; }
  return rank; // (multiple solutions if rank < m)
}
```

SolveLinear2.h

Description: To get all uniquely determined values of x back from SolveLinear, make the following changes: Define `undefined` as `numeric_limits<double>::quiet_NaN()` or another chosen sentinel.
Usage:

```
vec<vd>A={{1, 0}};vd b={2}, x(2);
int rank=solveLinear(A, b, x);
cout<<x[0]; // 2; x[1] is undefined, rank=1
```

"SolveLinear.h"

7 lines

```
rep(j, 0, n) if (j!=i) // instead of rep(j,i+1,n)
// ... then at the end:
x.assign(m, undefined);
rep(i, 0, rank) {
  rep(j, rank, m) if (fabs(A[i][j])>eps) goto fail;
  x[col[i]]=b[i]/A[i][i];
fail:; }
```

SolveLinearBinary.h

Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b.
Usage:
```
vec<bs>A(2);A[0].set(0);A[0].set(1);A[1].set(1);
vi b={1, 0};bs x;
int rank=solveLinear(A, b, x, 2);
if(rank!=-1)cout<<x[0]<<' '<<x[1]; // 1 0
```
Time: $\mathcal{O}\left(n^2m\right)$

20 lines

```
typedef bitset<1000>bs;
int solveLinear(vector<bs>&A, vi&b, bs&x, int m) {
  int n=sz(A), rank=0, br; assert(m<=sz(x));
  vi col(m); iota(all(col), 0);
  rep(i, 0, n) {
    for (br=i; br<n;++br) if (A[br].any()) break;
    if (br==n) { rep(j, i, n) if(b[j]) return-1;
      break; }
    int bc=(int)A[br]._Find_next(i-1); swap(A[i], A[br]);
    swap(b[i], b[br]); swap(col[i], col[bc]);
    rep(j, 0, n) if (A[j][i]!=A[j][bc]) {
      A[j].flip(i); A[j].flip(bc); }
    rep(j, i+1, n) if (A[j][i]) { b[j]^=b[i]; A[j]^=A[i]; }
    rank++; }
  x=bs();
  for (int i=rank; i--;) {
    if (!b[i]) continue;
    x[col[i]]=1; rep(j, 0, i) b[j]^=A[j][i]; }
  return rank; // (multiple solutions if rank < m)
}
```

MatrixInverse.h

Description: Invert matrix A. Returns rank; result is stored in A unless singular (rank < n). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1})$ (mod p^k) where A^{-1} starts as the inverse of A mod p, and k is doubled in each step.
Usage:
```
vec<vec<double>>A={{2, 0}, {0, 4}};
int rank=matInv(A);
if(rank==2)cout<<A[0][0]<<' '<<A[1][1]; // .5 .25
```
Time: $\mathcal{O}\left(n^3\right)$

19 lines

```
int matInv(vector<vector<double>>&A) { int n=sz(A); vi col(n);
  vector<vector<double>>tmp(n, vector<double>(n));
  rep(i, 0, n) tmp[i][i]=1, col[i]=i;
  rep(i, 0, n) { int r=i, c=i; rep(j, i, n) rep(k, i, n)
      if (fabs(A[j][k])>fabs(A[r][c]))
        r=j, c=k;
    if (fabs(A[r][c])<1e-12) return i;
    A[i].swap(A[r]); tmp[i].swap(tmp[r]); rep(j, 0, n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
    swap(col[i], col[c]); double v=A[i][i];
    rep(j, i+1, n) { double f=A[j][i]/v; A[j][i]=0;
      rep(k, i+1, n) A[j][k]-=f*A[i][k];
      rep(k, 0, n) tmp[j][k]-=f*tmp[i][k]; }
    rep(j, i+1, n) A[i][j]/=v; rep(j, 0, n) tmp[i][j]/=v;
    A[i][i]=1; }
  for (int i=n-1; i>0;--i) rep(j, 0, i) { double v=A[j][i];
    rep(k, 0, n) tmp[j][k]-=v*tmp[i][k]; }
  rep(i, 0, n) rep(j, 0, n) A[col[i]][col[j]]=tmp[i][j];
  return n; }
```

Tridiagonal.h

Description: $x = \text{tridiagonal}(d, p, q, b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i,\ 1 \le i \le n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, ..., -1, 1\}, \{0, c_1, c_2, \ldots, c_n\}, \{b_1, b_2, \ldots, b_n, 0\}, \{a_0, d_1, d_2, \ldots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.
If $|d_i| > |p_i| + |q_{i-1}|$ for all i, or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither `tr` nor the check for `diag[i] == 0` is needed.
Usage:
```
auto x=tridiagonal({2, 2}, {-1}, {-1}, {1, 1});
cout<<x[0]<<' '<<x[1]; // 1 1
```
Time: $\mathcal{O}\left(N\right)$

18 lines

```
typedef double T;
vector<T>tridiagonal(vector<T>diag, const vector<T>&super,
    const vector<T>&sub, vector<T>b) { int n=sz(b); vi tr(n);
  rep(i, 0, n-1) {
    if (abs(diag[i])<1e-9*abs(super[i])) { // diag[i] == 0
      b[i+1]-=b[i]*diag[i+1]/super[i];
      if (i+2<n) b[i+2]-=b[i]*sub[i+1]/super[i];
      diag[i+1]=sub[i]; tr[++i]=1;
    } else { diag[i+1]-=super[i]*sub[i]/diag[i];
      b[i+1]-=b[i]*sub[i]/diag[i]; }
  }
  for (int i=n; i--;) {
    if (tr[i]) { swap(b[i], b[i-1]);
      diag[i-1]=diag[i]; b[i]/=super[i-1];
    } else { b[i]/=diag[i];
      if (i) b[i-1]-=b[i]*super[i-1]; }
  }
  return b; }
```

4.4 Fourier transforms

FastFourierTransform.h

Description: fft(a) computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k. N must be a power of 2. Useful for convolution: `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n, reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16}; higher for random inputs). Otherwise, use NTT/FFTMod.
Usage:
```
vd a={1, 2}, b={3, 4};auto c=conv(a, b);
for(double x:c)cout<<llround(x)<<' '; // 3 10 8
```
Time: $\mathcal{O}\left(N \log N\right)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

23 lines

```
typedef complex<double>C; typedef vector<double>vd;
void fft(vector<C>&a) { int n=sz(a), L=31-__builtin_clz(n);
  static vector<complex<long double>>R(2, 1);
  static vector<C>rt(2, 1);  // (^ 10% faster if double)
  for (static int k=2; k<n; k*=2) { R.resize(n); rt.resize(n);
    auto x=polar(1.0L, acos(-1.0L)/k);
    rep(i, k, 2*k) rt[i]=R[i]=i&1?R[i/2]*x:R[i/2]; }
  vi rev(n); rep(i, 0, n) rev[i]=(rev[i/2]|(i&1)<<L)/2;
  rep(i, 0, n) if (i<rev[i]) swap(a[i], a[rev[i]]);
  for (int k=1; k<n; k*=2)
    for (int i=0; i<n; i+=2*k) rep(j, 0, k) {
      C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
      a[i+j+k]=a[i+j]-z; a[i+j]+=z; }
}
vd conv(const vd&a, const vd&b) {
  if (a.empty()||b.empty()) return {};
  vd res(sz(a)+sz(b)-1);
  int L=32-__builtin_clz(sz(res)), n=1<<L;
  vector<C>in(n), out(n); copy(all(a), begin(in));
  rep(i, 0, sz(b)) in[i].imag(b[i]); fft(in);
  for (C&x:in) x*=x;
  rep(i, 0, n) out[i]=in[-i&(n-1)]-conj(in[i]); fft(out);
  rep(i, 0, sz(res)) res[i]=imag(out[i])/(4*n); return res; }
```

FastFourierTransformMod.h

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.
Usage:
```
vl a={1, 2}, b={3, 4};auto c=convMod<1000000007>(a, b);
for(ll x:c)cout<<x<<' '; // 3 10 8
```
Time: $\mathcal{O}\left(N \log N\right)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

"FastFourierTransform.h" 18 lines

```
typedef vector<ll>vl;
template<int M>vl convMod(const vl&a, const vl&b) {
  if (a.empty()||b.empty()) return {};
  vl res(sz(a)+sz(b)-1);
  int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
  vector<C>L(n), R(n), outs(n), outl(n);
  rep(i, 0, sz(a)) L[i]=C((int)a[i]/cut, (int)a[i]%cut);
  rep(i, 0, sz(b)) R[i]=C((int)b[i]/cut, (int)b[i]%cut);
  fft(L), fft(R);
  rep(i, 0, n) { int j=-i&(n-1);
    outl[j]=(L[i]+conj(L[j]))*R[i]/(2.0*n);
    outs[j]=(L[i]-conj(L[j]))*R[i]/(2.0*n)/1i; }
  fft(outl), fft(outs);
  rep(i, 0, sz(res)) {
    ll av=ll(real(outl[i])+.5), cv=ll(imag(outs[i])+.5);
    ll bv=ll(imag(outl[i])+.5)+ll(real(outs[i])+.5);
    res[i]=((av%M*cut+bv)%M*cut+cv)%M; }
  return res; }
```

NumberTheoreticTransform.h

Description: ntt(a) computes $\hat{f}(k) = \sum_x a[x] g^{xk}$ for all k, where $g = \text{root}^{(mod-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^a b + 1$, where the convolution result has size at most 2^a. For arbitrary modulo, see FFTMod. `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by n, reverse(start+1, end), NTT back. Inputs must be in [0, mod). Use `ModPow` with the same modulus and retain only one definition of `mod`.
Usage:
```
vl a={1, 2}, b={3, 4};auto c=conv(a, b);
for(ll x:c)cout<<x<<' '; // 3 10 8
```
Time: $\mathcal{O}\left(N \log N\right)$

"../number-theory/ModPow.h" 24 lines

```
const ll mod=(119<<23)+1, root=62; // = 998244353
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). The last two are > 10^9.
typedef vector<ll>vl;
void ntt(vl&a) { int n=sz(a), L=31-__builtin_clz(n);
  static vl rt(2, 1);
  for (static int k=2, s=2; k<n; k*=2, s++) { rt.resize(n);
    ll z[]={1, modpow(root, mod>>s)};
    rep(i, k, 2*k) rt[i]=rt[i/2]*z[i&1]%mod; }
  vi rev(n); rep(i, 0, n) rev[i]=(rev[i/2]|(i&1)<<L)/2;
  rep(i, 0, n) if (i<rev[i]) swap(a[i], a[rev[i]]);
  for (int k=1; k<n; k*=2)
```

```
    for (int i=0; i<n; i+=2*k) rep(j, 0, k) {
      ll z=rt[j+k]*a[i+j+k]%mod, &ai=a[i+j];
      a[i+j+k]=ai-z+(z>ai?mod:0); ai+=(ai+z>=mod?z-mod:z); }
}
vl conv(const vl&a, const vl&b) {
  if (a.empty()||b.empty()) return {};
  int s=sz(a)+sz(b)-1, B=32-__builtin_clz(s),
      n=1<<B; int inv=modpow(n, mod-2); vl L(a), R(b), out(n);
  L.resize(n), R.resize(n); ntt(L), ntt(R); rep(i, 0, n)
    out[-i&(n-1)]=(ll)L[i]*R[i]%mod*inv%mod; ntt(out);
  return {out.begin(), out.begin()+s};
}
```

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x\oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.

Usage:

```
vi a={1, 2}, b={3, 4};auto c=conv(a, b);
cout<<c[0]<<' '<<c[1]; // 13 8, AND convolution
```

Time: $\mathcal{O}(N \log N)$

13 lines

```
void FST(vi&a, bool inv) {
  for (int n=sz(a), step=1; step<n; step*=2) {
    for (int i=0; i<n; i+=2*step) rep(j, i, i+step) {
      int&u=a[j], &v=a[j+step]; tie(u, v)=
        inv?pii(v-u, u):pii(v, u+v); // AND
        inv ? pii(v, u - v) : pii(u + v, u); // OR
        pii(u+v, u-v);                   // XOR
    }
  }
  if (inv) for (int&x:a) x/=sz(a); // XOR only
}
vi conv(vi a, vi b) { FST(a, 0); FST(b, 0);
  rep(i, 0, sz(a)) a[i]*=b[i]; FST(a, 1); return a; }
```

Number theory (5)

5.1 Modular arithmetic

ModularArithmetic.h

Description: Operators for modular arithmetic. You need to set `mod` to some number first and then you can use the structure.

Usage:

```
Mod a(8), b(6);Mod sum=a+b, quotient=a/b;
cout<<sum.x<<' '<<quotient.x; // 14 7
```

"euclid.h"

14 lines

```
const ll mod=17; // change to something else
struct Mod { ll x;
  Mod(ll y):Mod(y%mod+mod, 0){}
  Mod(ll y, int):x(y<mod?y:y-mod){}
  Mod operator+(Mod b) { return {x+b.x, 0}; }
  Mod operator-(Mod b) { return {x-b.x+mod, 0}; }
  Mod operator*(Mod b) { return {x*b.x%mod, 0}; }
  Mod operator/(Mod b) { return*this*invert(b); }
  Mod invert(Mod a) { ll x, y, g=euclid(a.x, mod, x, y);
    assert(g==1); return x; }
  Mod operator^(ll e) {
    if (!e) return 1;
    Mod r=*this^(e/2); r=r*r; return e&1?*this*r:r; }
};
```

ModInverse.h

Description: Pre-computation of modular inverses. Assumes LIM $\leq$ mod and that mod is a prime. Paste the precomputation inside a function after defining `mod` and `LIM`.

Usage:

```
const ll mod=7, LIM=7;
// Paste fragment here.
cout<<inv[3]; // 5, because 3*5=1 mod 7
```

3 lines

```
const ll mod=1000000007, LIM=200000;
ll*inv=new ll[LIM]-1; inv[1]=1;
rep(i, 2, LIM) inv[i]=mod-(mod/i)*inv[mod%i]%mod;
```

ModPow.h

Usage:

```
ll result=modpow(2, 10);cout<<result; // 1024
```

5 lines

```
const ll mod=1000000007; // faster if const
ll modpow(ll b, ll e) { ll ans=1;
  for (; e; b=b*b%mod, e/=2)
    if (e&1) ans=ans*b%mod;
  return ans; }
```

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a.

Usage:

```
ll exponent=modLog(2, 8, 13);
cout<<exponent; // 3
```

Time: $\mathcal{O}\left(\sqrt{m}\right)$

8 lines

```
ll modLog(ll a, ll b, ll m) {
  ll n=(ll) sqrt(m)+1, e=1, f=1, j=1; unordered_map<ll, ll>A;
  while (j<=n&&(e=f=e*a%m)!=b%m)
    A[e*b%m]=j++;
  if (e==b%m) return j;
  if (__gcd(m, e)==__gcd(m, b))
    rep(i, 2, n+2) if (A.count(e=e*f%m))
      return n*i-A[e]; return-1; }
```

ModSum.h

Description: Sums of mod'ed arithmetic progressions.

`modsum(to, c, k, m)` $= \sum_{i=0}^{\text{to}-1} (ki + c)\%m$. `divsum` is similar but for floored division.

Usage:

```
cout<<modsum(4, 1, 2, 5); // 1+3+0+2 = 6
cout<<divsum(4, 1, 2, 5); // 0+0+1+1 = 2
```

Time: $\log(m)$, with a large constant.

10 lines

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to/2*((to-1)|1); }
ull divsum(ull to, ull c, ull k, ull m) {
  ull res=k/m*sumsq(to)+c/m*to; k%=m; c%=m;
  if (!k) return res;
  ull to2=(to*k+c)/m;
  return res+(to-1)*to2-divsum(to2, m-1-c, m, k); }
ll modsum(ull to, ll c, ll k, ll m) { c=((c%m)+m)%m;
  k=((k%m)+m)%m;
  return to*c+k*sumsq(to)-m*divsum(to, c, k, m); }
```

ModMulLL.h

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.

Usage:

```
cout<<modmul(7, 8, 13); // 4
cout<<modpow(2, 10, 13); // 10
```

Time: $\mathcal{O}(1)$ for `modmul`, $\mathcal{O}(\log b)$ for `modpow`

7 lines

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) { ll ret=a*b-M*ull(1.L/M*a*b);
  return ret+M*(ret<0)-M*(ret>=(ll)M); }
ull modpow(ull b, ull e, ull mod) { ull ans=1;
  for (; e; b=modmul(b, b, mod), e/=2)
    if (e&1) ans=modmul(ans, b, mod);
  return ans; }
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution). Use the three-argument `modpow(b, e, p)` from `ModMulLL.h` instead of the fixed-modulus dependency.

Usage:

```
ll r=sqrt(4LL, 7LL);cout<<(r*r%7); // 4
```

Time: $\mathcal{O}\left(\log^2 p\right)$ worst case, $\mathcal{O}(\log p)$ for most p

"ModPow.h"

18 lines

```
ll sqrt(ll a, ll p) { a%=p; if (a<0) a+=p;
  if (a==0) return 0;
  assert(modpow(a, (p-1)/2, p)==1); // else no solution
  if (p%4==3) return modpow(a, (p+1)/4, p);
  // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
  ll s=p-1, n=2; int r=0, m;
  while (s%2==0)
    ++r, s/=2;
  while (modpow(n, (p-1)/2, p)!=p-1)++n;
  ll x=modpow(a, (s+1)/2, p);
  ll b=modpow(a, s, p), g=modpow(n, s, p);
  for (;; r=m) { ll t=b;
    for (m=0; m<r&&t!=1;++m)
      t=t*t%p;
    if (m==0) return x;
    ll gs=modpow(g, 1LL<<(r-m-1), p); g=gs*gs%p; x=x*gs%p;
    b=b*g%p; }
}
```

5.2 Primality

MathSieve.h

Description: Euler linear sieve for primes, smallest prime factors and factorization within [1,n]. Optional phi, mu and divisor-count modules remain commented out; enable their declarations, initialization and recurrences together. All products p*i must fit int.

Usage:

```
MathSieve sieve(1000);cout<<(sieve.spf[97]==97); // 1
for(auto [p, e]:sieve.factorize(120))
cout<<p<<'^'<<e<<' '; // 2^3 3^1 5^1
```

Time: $\mathcal{O}(N)$ build and space; $\mathcal{O}(\log x)$ factorization.

32 lines

```
struct MathSieve { int n; std::vec<int>primes, spf;
// std::vec<ll>phi;
// std::vec<int>mu;
// std::vec<int>d, cnt;
  MathSieve(int _n):n(_n), spf(_n+1) /*, phi(_n + 1), mu(_n +
      1), d(_n + 1), cnt(_n + 1)*/ {
// [Phi] if (_n>=1) phi[1]=1;
// [Mu] if (_n>=1) mu[1]=1;
// [Divisor count] if (_n>=1) { d[1]=1; cnt[1]=0; }
    for (int i=2; i<=n;++i) spf[i]=i;
    for (int i=2; i<=n;++i) { if (spf[i]==i) {
        primes.push_back(i);
// [Phi] phi[i]=i-1;
// [Mu] mu[i]=-1;
// [Divisor count] d[i]=2; cnt[i]=1;
      }
      for (int p:primes) { if (p*i>n) break;
        spf[p*i]=p; if (i%p==0) {
// [Phi] phi[p*i]=phi[i]*p;
// [Mu] mu[p*i]=0;
// [Divisor count] d[p * i] = d[i] / (cnt[i] + 1) * (cnt[i] +
// 2); cnt[p * i] = cnt[i] + 1;
          break; } else {
// [Phi] phi[p*i]=phi[i]*phi[p];
// [Mu] mu[p*i]=-mu[i];
```

```
// [Divisor count] d[p*i]=d[i]*d[p]; cnt[p*i]=1;
        } } } }
  std::vec<std::pair<int, int>>factorize(int x) {
    std::vec<std::pair<int, int>>factors; while (x>1) {
      int p=spf[x]; int count=0; while (x%p==0) {
        count++; x/=p; }
        factors.push_back({p, count}); }
    return factors; } };
```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Usage: `cout<<isPrime(97)<<' '<<isPrime(100); // 1 0`
Time: 7 times the complexity of $a^b \mod c$.

"ModMulLL.h" — 10 lines

```
bool isPrime(ull n) {
  if (n<2||n%6%4!=1) return (n|1)==3;
  ull A[]={2, 325, 9375, 28178, 450775, 9780504, 1795265022},
      s=__builtin_ctzll(n-1), d=n>>s;
  for (ull a:A) {   // ^ count trailing zeroes
    ull p=modpow(a%n, d, n), i=s;
    while (p!=1&&p!=n-1&&a%n&&i--)
      p=modmul(p, p, n);
    if (p!=n-1&&i!=s) return 0; }
  return 1; }
```

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Usage:

```
auto primes=factor(120);sort(all(primes));
for(auto p:primes)cout<<p<<' '; // 2 2 2 3 5
```

Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMulLL.h", "MillerRabin.h" — 12 lines

```
ull pollard(ull n) { ull x=0, y=0, t=30, prd=2, i=1, q;
  auto f=[&](ull x) { return modmul(x, x, n)+i; };
  while (t++%40||__gcd(prd, n)==1) {
    if (x==y) x=++i, y=f(x);
    if ((q=modmul(prd, max(x, y)-min(x, y), n))) prd=q;
    x=f(x), y=f(f(y)); }
  return __gcd(prd, n); }
vector<ull>factor(ull n) {
  if (n==1) return {};
  if (isPrime(n)) return {n};
  ull x=pollard(n); auto l=factor(x), r=factor(n/x);
  l.insert(l.end(), all(r)); return l; }
```

5.3 Divisibility

euclid.h

Description: Finds two integers x and y, such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in __gcd instead. If a and b are coprime, then x is the inverse of $a \pmod b$.
Usage:

```
ll a=6, b=9, c=3, x, y;ll g=euclid(a, b, x, y);
if(c%g)cout<<-1;
else { // ax+by=c; here a,b>0
ll step=b/g;
x=(__int128)x*(c/g)%step;x=(x+step)%step;
if(!x)x+=step;
y=(c-(__int128)a*x)/b;
cout<<x<<' '<<y; // 2 -1; smallest positive x
}
```

3 lines

```
ll euclid(ll a, ll b, ll&x, ll&y) {
  if (!b) return x=1, y=0, a;
  ll d=euclid(b, a%b, y, x); return y-=a/b*x, d; }
```

CRT.h

Description: Chinese Remainder Theorem.
`crt(a, m, b, n)` computes x such that $x \equiv a \pmod m$, $x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \le x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
Usage:

```
ll a=2, m=6, b=8, n=9;
if((a-b)%gcd(m, n))cout<<-1;
else cout<<crt(a, m, b, n)<<' '<<lcm(m, n); // 8 18
```

Time: $\log(n)$

"euclid.h" — 5 lines

```
ll crt(ll a, ll m, ll b, ll n) {
  if (n>m) swap(a, b), swap(m, n);
  ll x, y, g=euclid(m, n, x, y);
  assert((a-b)%g==0); // else no solution
  x=(b-a)%n*x%n/g*m+a; return x<0?x+m*n/g:x; }
```

5.3.1 Bézout's identity

For $a \neq, b \neq 0$, then $d = gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a, b)}, y - \frac{ka}{\gcd(a, b)}\right), \quad k \in \mathbb{Z}$$

BigInt.h

Description: Nonnegative base-1e9 integers with arithmetic, comparisons and streams. Subtraction requires a >= b. Construct numeric zero as BigInt(0). Scalar operands should not exceed 2e9; division/modulo require a positive divisor.
Usage:

```
BigInt a(string("100000000000000000")), b(7);
BigInt c=a+b;cout<<c; // 100000000000000007
cout<<(a-b<a); // 1
```

Time: $\mathcal{O}(D)$ add/subtract, $\mathcal{O}\left(D^2\right)$ multiply, $\mathcal{O}(D)$ space.

75 lines

```
struct BigInt { static const int BASE=1e9; vec<int>v;
  BigInt() {}
  BigInt(ll val) { if (val==0) v.push_back(0);
    while (val>0) { v.push_back(val%BASE); val/=BASE; } }
  BigInt(string s) {
    if (s.empty()||s=="0") { v.push_back(0); return; }
    for (int i=(int)s.length(); i>0; i-=9) {
      if (i<9) v.push_back(stoi(s.substr(0, i)));
      else v.push_back(stoi(s.substr(i-9, 9))); }
      trim(); }
  void trim() {
    while (v.size()>1&&v.back()==0) v.pop_back(); }
  bool operator<(const BigInt&b) const {
    if (v.size()!=b.v.size()) return v.size()<b.v.size();
    for (int i=(int)v.size()-1; i>=0; i--) {
      if (v[i]!=b.v[i]) return v[i]<b.v[i]; }
      return false; }
  bool operator>(const BigInt&b) const { return b<
*this; }
  bool operator<=(const BigInt&b) const { return!(b<
*this); }
bool operator>=(const BigInt&b) const { return!
(*this<b); }
  bool operator==(const BigInt&b) const { return!
(*this<b)&&!(b<*this); }
  bool operator!=(const BigInt&b) const { return
(*this<b)||(b<*this); }
  BigInt operator+(const BigInt&b) const { BigInt res;
    int n=max(v.size(), b.v.size()); ll carry=0;
    for (int i=0; i<n||carry; i++) { ll sum=carry;
      if (i<v.size()) sum+=v[i];
      if (i<b.v.size()) sum+=b.v[i];
      res.v.push_back(sum%BASE); carry=sum/BASE; }
    res.trim(); return res; }
  BigInt operator-(const BigInt&b) const { BigInt res;
    ll borrow=0; for (int i=0; i<v.size(); i++) {
      ll diff=v[i]-borrow-(i<b.v.size()?b.v[i]:0);
      if (diff<0) { diff+=BASE; borrow=1; }
      else borrow=0; res.v.push_back(diff); }
      res.trim(); return res; }
    BigInt operator*(const BigInt&b) const {
    if ((v.size()==1&&v[0]==0)||(b.v.size()==
1&&b.v[0]==0)) return BigInt(0); BigInt res;
    res.v.assign(v.size()+b.v.size(), 0);
    for (int i=0; i<v.size(); i++) { ll carry=0;
      for (int j=0; j<b.v.size()||carry; j++) {
        ll cur=res.v[i+j]+1LL*v[i]*
(j<b.v.size()?b.v[j]:0)+carry;
        res.v[i+j]=cur%BASE; carry=cur/BASE; } }
    res.trim(); return res; }
  BigInt operator*(const ll&b) const {
    if (b==0||(v.size()==1&&v[0]==0)) return
BigInt(0); BigInt res; ll carry=0;
    for (int i=0; i<v.size()||carry; i++) {
      ll cur=carry; if (i<v.size()) cur+=1LL*v[i]*b;
      res.v.push_back(cur%BASE); carry=cur/BASE; }
    res.trim(); return res; }
  BigInt operator/(const ll&b) const { BigInt res;
    res.v.assign(v.size(), 0); ll rem=0;
    for (int i=(int)v.size()-1; i>=0; i--) {
      rem=rem*BASE+v[i]; res.v[i]=rem/b; rem%=b; }
    res.trim(); return res; }
  ll operator%(const ll&b) const { ll rem=0;
    for (int i=(int)v.size()-1; i>=0; i--) {
      rem=(rem*BASE+v[i])%b; }
      return rem; }
  friend istream&operator>>(istream&is, BigInt&a) {
    string s; is>>s; a=BigInt(s); return is; }
  friend ostream&operator<<(ostream&os, const BigInt&a) {
    if (a.v.empty()) { os<<0; return os; }
    os<<a.v.back();
    for (int i=(int)a.v.size()-2; i>=0; i--) {
      os<<setfill('0')<<setw(9)<<a.v[i]; }
      return os; }
};
```

5.4 Fractions

ContinuedFractions.h

Description: Given N and a real number $x \ge 0$, finds the closest rational approximation p/q with $p, q \le N$. It will obey $|p/q - x| \le 1/qN$.
For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. ($p_k/q_k$ alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞; if x is the root of a degree 2 polynomial the a's eventually become cyclic.
Usage:

```
auto [p, q]=approximate(3.141592653589793, 1000);
cout<<p<<'/'<<q; // 355/113
```

Time: $\mathcal{O}(\log N)$

17 lines

```
typedef double d; // for N ~ 1e7; long double for N ~ 1e9
pair<ll, ll>approximate(d x, ll N) {
  ll LP=0, LQ=1, P=1, Q=0, inf=LLONG_MAX; d y=x;
```

```
  for (;;) { ll lim=min(P?(N-LP)/P:inf, Q?(N-LQ)/Q:inf),
       a=(ll)floor(y), b=min(a, lim),
       NP=b*P+LP, NQ=b*Q+LQ;
    if (a>b) {
      // If b > a/2, we have a semi-convergent that gives us a
      // better approximation; if b = a/2, we *may* have one.
      // Return {P, Q} here for a more canonical approximation.
      return (abs(x-(d)NP/(d)NQ)<abs(x-(d)P/(d)Q))?
        make_pair(NP, NQ):make_pair(P, Q); }
    if (abs(y=1/(y-(d)a))>3*N) {
      return {NP, NQ};
    }
    LP=P; P=NP; LQ=Q; Q=NQ; }
}
```

FracBinarySearch.h

Description: Given f and N, finds the smallest fraction $p/q \in [0,1]$ such that $f(p/q)$ is true, and $p, q \le N$. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed.

Usage:

```
Frac r=fracBS([](Frac f){return 3*f.p>=f.q;}, 10);
cout<<r.p<<'/'<<r.q; // 1/3
```

Time: $\mathcal{O}(\log(N))$

15 lines

```
struct Frac { ll p, q; };
template<class F>
Frac fracBS(F f, ll N) { bool dir=1, A=1, B=1;
  Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
  if (f(lo)) return lo;
  assert(f(hi));
  while (A||B) { ll adv=0, step=1; // move hi if dir, else lo
    for (int si=0; step; (step*=2)>>=si) { adv+=step;
      Frac mid{lo.p*adv+hi.p, lo.q*adv+hi.q};
      if (abs(mid.p)>N||mid.q>N||dir==!f(mid)) {
        adv-=step; si=2; }
    }
    hi.p+=lo.p*adv; hi.q+=lo.q*adv; dir=!dir; swap(lo, hi);
    A=B; B=!!adv; }
  return dir?hi:lo; }
```

5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2),\ \ b = k \cdot (2mn),\ \ c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

5.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a, except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.7 Estimates

$\sum_{d|n} d = O(n \log \log n)$.

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

5.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g(n/d)$$

Other useful formulas/forms:

$\sum_{d|n} \mu(d) = [n = 1]$ (very useful)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n) g(d)$

$g(n) = \sum_{1 \le m \le n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \le m \le n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$

Combinatorial (6)

Combination.h

Description: Precomputes factorials and inverse factorials for binomial coefficients modulo prime MOD. Requires 0 <= n < MAXN <= MOD. Invalid m returns 0.

Usage:

```
init_comb();cout<<C(5, 2); // 10
cout<<fac[5]; // 120
```

Time: $\mathcal{O}(MAXN + \log MOD)$ preprocessing, $\mathcal{O}(1)$ query, $\mathcal{O}(MAXN)$ space.

14 lines

```
const int MOD=1000000007; const int MAXN=1000005;
ll fac[MAXN]; ll invFac[MAXN]; ll qpow(ll base, ll exp) {
  ll res=1; base%=MOD; while (exp>0) {
    if (exp&1) res=res*base%MOD;
    base=base*base%MOD; exp>>=1; }
    return res; }
void init_comb() { fac[0]=1; invFac[0]=1;
  for (int i=1; i<MAXN; i++) { fac[i]=fac[i-1]*i%MOD; }
  invFac[MAXN-1]=qpow(fac[MAXN-1], MOD-2);
  for (int i=MAXN-2; i>=1; i--) {
    invFac[i]=invFac[i+1]*(i+1)%MOD; } }
ll C(ll n, ll m) { if (m>n||m<0) return 0;
  if (m==0||m==n) return 1;
  return fac[n]*invFac[m]%MOD*invFac[n-m]%MOD; }
```

6.1 Permutations

6.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

IntPerm.h

Description: Permutation -> integer conversion. (Not order preserving.) Integer -> permutation can use a lookup table.

Usage:

```
vi p={0, 1, 2};int code=permToInt(p);
cout<<code; // 0; p is unchanged
```

Time: $\mathcal{O}(n)$

4 lines

```
int permToInt(vi&v) { int use=0, i=0, r=0;
  for(int x:v) r=r*++i+__builtin_popcount(use&-(1<<x)),
    use|=1<<x;                    // (note: minus, not ~!)
  return r; }
```

6.1.2 Cycles

Let $g_S(n)$ be the number of n-permutations whose cycle lengths all belong to the set S. Then

$$\sum_{n=0}^{\infty} g_S(n) \frac{x^n}{n!} = \exp\left(\sum_{n \in S} \frac{x^n}{n} \right)$$

6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

6.2 Partitions and subsets

6.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1,\ p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	~2e5	~2e8

6.2.2 Lucas' Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + \ldots + n_1 p + n_0$ and $m = m_k p^k + \ldots + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^{k} \binom{n_i}{m_i} \pmod{p}$.

6.2.3 Binomials

multinomial.h

Description: Computes $\binom{k_1 + \cdots + k_n}{k_1, k_2, \ldots, k_n} = \frac{(\sum k_i)!}{k_1! k_2! \ldots k_n!}$.

Usage:

```
vi counts={2, 1, 1};ll ways=multinomial(counts);
cout<<ways; // 12
```

2 lines

```
ll multinomial(vi&v) { ll c=1, m=v.empty()?1:v[0];
  rep(i, 1, sz(v)) rep(j, 0, v[i]) c=c*++m/(j+1); return c; }
```

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t-1}$ (FFT-able).
$B[0,\dots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \dots]$

Sums of powers:

$$\sum_{i=1}^{n} n^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k}B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x)dx - \sum_{k=1}^{\infty}\frac{B_k}{k!}f^{(k-1)}(m)$$

$$\approx \int_m^{\infty} f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

6.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$
$$\textstyle\sum_{k=0}^{n} c(n,k)x^k = x(x+1)\dots(x+n-1)$$

$c(8,k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \dots$

6.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j:s s.t. $\pi(j) \geq j$, k j:s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$

$$E(n,0) = E(n,n-1) = 1$$

$$E(n,k) = \sum_{j=0}^{k}(-1)^j\binom{n+1}{j}(k+1-j)^n$$

6.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$

$$S(n,1) = S(n,n) = 1$$

$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$$

6.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod p$$

6.3.6 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i: $n_1 n_2 \cdots n_k n^{k-2}$
with degrees d_i: $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

6.3.7 Catalan numbers

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Graph (7)

7.1 Fundamentals

ZeroOneBFS.h
Description: Directed shortest paths for weights 0 and 1. Add both arcs for an undirected edge. run(s) resets distances and parents. Unreachable distance is INF, path is empty. Adding an edge invalidates paths until run is called again.
Usage:
```
ZeroOneBFS bfs(3);bfs.add_edge(0, 1, 0);
bfs.add_edge(1, 2, 1);bfs.run(0);
cout<<bfs.dist[2]; // 1
for(int v:bfs.get_path(2))cout<<v; // 012
```
Time: $\mathcal{O}(V+E)$ per run, $\mathcal{O}(V+E)$ space.
24 lines

```
struct ZeroOneBFS { static constexpr int INF=INT_MAX;
 int n, source=-1; vec<vec<pair<int, int>>>adj;
 vec<int>dist, parent; explicit ZeroOneBFS(int N):n(N) {
  if (n<0) throw invalid_argument("negative size");
  adj.resize(n); dist.assign(n, INF); parent.assign(n, -1); }
 void check(int v) const {
  if (v<0||v>=n) throw out_of_range("vertex"); }
 void add_edge(int u, int v, int w) { check(u); check(v);
  if (w!=0&&w!=1) throw invalid_argument("weight");
  adj[u].push_back({v, w}); source=-1; }
 void run(int s) {
  check(s); source=s; fill(dist.begin(), dist.end(), INF);
  fill(parent.begin(), parent.end(), -1); vec<bool>done(n);
  deque<int>q; q.push_back(s); dist[s]=0; while (!q.empty()) {
   int u=q.front(); q.pop_front(); if (done[u]) continue;
   done[u]=true;
   for (auto [v, w]:adj[u]) if (dist[u]+w<dist[v]) {
  dist[v]=dist[u]+w; parent[v]=u;
  if (w) q.push_back(v); else q.push_front(v); } } }
 vec<int>get_path(int t) const { check(t);
  if (source<0||dist[t]==INF) return {};
  vec<int>p;
  for (int v=t;v!=-1;v=parent[v]) p.push_back(v);
  reverse(p.begin(), p.end()); return p; } };
```

BellmanFord.h
Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get dist = inf; nodes reachable through negative-weight cycles get dist = -inf. Assumes $V^2 \max|w_i| < \sim 2^{63}$.
Usage:
```
vec<Node>nodes(3);vec<Ed>edges={{0, 1, 2}, {1, 2, -1}};
bellmanFord(nodes, edges, 0);
cout<<nodes[2].dist<<' '<<nodes[2].prev; // 1 1
```
Time: $\mathcal{O}(VE)$
18 lines

```
const ll inf=LLONG_MAX;
struct Ed { int a, b, w, s() { return a<b?a:-a; }};
struct Node { ll dist=inf; int prev=-1; };
void bellmanFord(vector<Node>&nodes, vector<Ed>&eds, int s) {
  nodes[s].dist=0;
  sort(all(eds), [](Ed a, Ed b) { return a.s()<b.s(); });
  int lim=sz(nodes)/2+2; // /3+100 with shuffled vertices
  rep(i, 0, lim) for (Ed ed:eds) {
    Node cur=nodes[ed.a], &dest=nodes[ed.b];
    if (abs(cur.dist)==inf) continue;
    ll d=cur.dist+ed.w;
    if (d<dest.dist) { dest.prev=ed.a;
      dest.dist=(i<lim-1?d:-inf); }
  }
  rep(i, 0, lim) for (Ed e:eds) {
    if (nodes[e.a].dist==-inf)
      nodes[e.b].dist=-inf; }
}
```

7.2 Network flow

Dinic.h
Description: Integer max-flow and s-t min-cut; capacities are nonnegative and $s \neq t$. The fourth `addEdge` argument is reverse capacity (default 0; use c for undirected edges). `calc` returns additional flow and updates residual capacities c; signed edge flow is `oc-c`. Read the source side with `leftOfMinCut` after `calc`.
For bipartite matching, `g[u]` lists right IDs $[0, m)$ and `r.size()=m`. `dfsMatching` keeps `g` unchanged, returns the size and fills `r[right]=left` (-1 if unmatched). `hopcroftKarp` is an alias. `cover(g,n,m)` returns left IDs u and right IDs $n+v$; its complement is a maximum independent set.
Usage:
```
// Max flow / min cut
Dinic f(3);
f.addEdge(0, 1, 4);f.addEdge(1, 2, 3);
cout<<f.calc(0, 2); // max flow:  3
cout<<f.adj[0][0].flow(); // edge 0->1 flow:  3
cout<<f.leftOfMinCut(0); // source side:  1
cout<<f.leftOfMinCut(2); // source side:  0
// Bipartite matching:  2 left, 2 right
vec<vi>g={{0, 1}, {1}};vi r(2, -1);
cout<<dfsMatching(g, r); // size:  2
cout<<r[0]<<' '<<r[1]; // right->left:  0 1
// Minimum vertex cover / maximum independent set
vi cv=cover(g, 2, 2);
for(int v:cv)cout<<v<<' '; // cover:  0 1
```

```
vi used(4);for(int v:cv)used[v]=1;
rep(v, 0, 4)if(!used[v])cout<<v<<' '; // independent:  2 3
```

Time: $\mathcal{O}(VE\log U)$, U=max capacity; $\mathcal{O}\left((E+V)\sqrt{V}\right)$ for matching/cover.

43 lines

```
struct Dinic {
 struct Edge { int to, rev;ll c, oc;
  ll flow(){return max(oc-c, 0LL);} };
 vi lvl, ptr, q;vec<vec<Edge>>adj;
 Dinic(int n):lvl(n), ptr(n), q(n), adj(n){}
 void addEdge(int a, int b, ll c, ll rcap=0){
  adj[a].push_back({b, sz(adj[b]), c, c});
  adj[b].push_back({a, sz(adj[a])-1, rcap, rcap}); }
 ll dfs(int v, int t, ll f){if(v==t||!f)return f;
  for(int&i=ptr[v];i<sz(adj[v]);i++){Edge&e=adj[v][i];
   if(lvl[e.to]==lvl[v]+1)
    if(ll p=dfs(e.to, t, min(f, e.c))){
     e.c-=p, adj[e.to][e.rev].c+=p;return p; }
  }
  return 0; }
 ll calc(int s, int t){ll flow=0;q[0]=s;
  rep(L, 0, 31)do{ // 'int L=30' maybe faster for random data
   lvl=ptr=vi(sz(q));int qi=0, qe=lvl[s]=1;
   while(qi<qe&&!lvl[t]){int v=q[qi++];
    for(Edge e:adj[v])if(!lvl[e.to]&&e.c>>(30-L))
     q[qe++]=e.to, lvl[e.to]=lvl[v]+1; }
   while(ll p=dfs(s, t, LLONG_MAX))flow+=p;
  }while(lvl[t]);return flow; }
 bool leftOfMinCut(int a){return lvl[a]!=0;}
};
namespace dinic_detail {
Dinic bipartite(const vec<vi>&g, int m){
 int n=sz(g), s=n+m, t=s+1;Dinic f(t+1);
 rep(u, 0, n){f.addEdge(s, u, 1);
  for(int v:g[u])f.addEdge(u, n+v, 1);}
 rep(v, 0, m)f.addEdge(n+v, t, 1);f.calc(s, t);return f; }}
int dfsMatching(vec<vi>&g, vi&btoa){
 int n=sz(g), m=sz(btoa), res=0;
 auto f=dinic_detail::bipartite(g, m);fill(all(btoa), -1);
 rep(u, 0, n)for(auto&e:f.adj[u])
  if(e.to>=n&&e.to<n+m&&e.flow())btoa[e.to-n]=u, ++res;
 return res; }
int hopcroftKarp(vec<vi>&g, vi&r){return dfsMatching(g, r);}
vi cover(vec<vi>&g, int n, int m){
 assert(n==sz(g));auto f=dinic_detail::bipartite(g, m);vi ans;
 rep(u, 0, n)if(!f.leftOfMinCut(u))ans.push_back(u);
 rep(v, 0, m)if(f.leftOfMinCut(n+v))ans.push_back(n+v);
 return ans; }
```

MinCostMaxFlow.h

Description: Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only. Also include `bits/extc++.h` for the PBDS queue.

Usage:

```
MCMF mf(3);mf.addEdge(0, 1, 2, 3);mf.addEdge(1, 2, 2, 1);
auto [flow, cost]=mf.maxflow(0, 2);
cout<<flow<<' '<<cost; // 2 8
cout<<mf.ed[0][0].flow; // 2
```

Time: $\mathcal{O}(FE\log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi.

51 lines

```
#include <bits/extc++.h>
const ll INF=numeric_limits<ll>::max()/4;
struct MCMF {
  struct edge { int from, to, rev; ll cap, cost, flow; };
  int N; vector<vector<edge>>ed; vi seen; vector<ll>dist, pi;
  vector<edge*>par;
  MCMF(int N):N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}
  void addEdge(int from, int to, ll cap, ll cost) {
    if (from==to) return;
    ed[from].push_back(edge{ from, to, sz(ed[to]), cap, cost, 0
        });
    ed[to].push_back(edge{ to, from, sz(ed[from])-1, 0, -cost,
        0 });
  }
  void path(int s) { fill(all(seen), 0); fill(all(dist), INF);
    dist[s]=0; ll di;
    __gnu_pbds::priority_queue<pair<ll, int>>q;
    vector<decltype(q)::point_iterator>its(N);
    q.push({ 0, s });
    while (!q.empty()) { s=q.top().second; q.pop();
      seen[s]=1; di=dist[s]+pi[s];
      for (edge&e:ed[s]) if (!seen[e.to]) {
        ll val=di-pi[e.to]+e.cost;
        if (e.cap-e.flow>0&&val<dist[e.to]) { dist[e.to]=val;
          par[e.to]=&e;
          if (its[e.to]==q.end())
            its[e.to]=q.push({-dist[e.to], e.to });
          else
            q.modify(its[e.to], {-dist[e.to], e.to }); }
      }
    }
    rep(i, 0, N) pi[i]=min(pi[i]+dist[i], INF); }
  pair<ll, ll>maxflow(int s, int t) { ll totflow=0, totcost=0;
    while (path(s), seen[t]) { ll fl=INF;
      for (edge*x=par[t]; x; x=par[x->from])
        fl=min(fl, x->cap-x->flow); totflow+=fl;
      for (edge*x=par[t]; x; x=par[x->from]) { x->flow+=fl;
        ed[x->to][x->rev].flow-=fl; }
    }
    rep(i, 0, N) for(edge&e:ed[i]) totcost+=e.cost*e.flow;
    return {totflow, totcost/2};
  }
  // If some costs can be negative, call this before maxflow:
  void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF); pi[s]=0; int it=N, ch=1; ll v;
    while (ch--&&it--)
      rep(i, 0, N) if (pi[i]!=INF)
        for (edge&e:ed[i]) if (e.cap)
          if ((v=pi[i]+e.cost)<pi[e.to])
            pi[e.to]=v, ch=1;
    assert(it>=0); // negative cost cycle
  }
};
```

GlobalMinCut.h

Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.

Usage:

```
vec<vi>m={{0, 3, 1}, {3, 0, 2}, {1, 2, 0}};
auto [value, side]=globalMinCut(m);cout<<value; // 3
for(int v:side)cout<<v<<' '; // cut-side vertices
```

Time: $\mathcal{O}(V^3)$

13 lines

```
pair<int, vi>globalMinCut(vector<vi>mat) {
  pair<int, vi>best={INT_MAX, {}};
  int n=sz(mat); vector<vi>co(n);
  rep(i, 0, n) co[i]={i};
  rep(ph, 1, n) { vi w=mat[0]; size_t s=0, t=0;
    rep(it, 0, n-ph) { // O(V^2) -> O(E log V) with prio. queue
      w[t]=INT_MIN; s=t, t=max_element(all(w))-w.begin();
      rep(i, 0, n) w[i]+=mat[t][i]; }
    best=min(best, {w[t]-mat[t][t], co[t]});
    co[s].insert(co[s].end(), all(co[t]));
    rep(i, 0, n) mat[s][i]+=mat[t][i];
    rep(i, 0, n) mat[i][s]=mat[s][i]; mat[0][t]=INT_MIN; }
  return best; }
```

7.3 Matching

WeightedMatching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes cost[N][M], where cost[i][j] = cost for L[i] to be matched with R[j], and returns (min cost, match), where L[i] is matched with R[match[i]]. Negate costs for max cost. Requires $N \leq M$.

Usage:

```
vec<vec<ll>>cost={{1, 5}, {4, 2}};
auto [value, match]=weightedMatching(cost);
cout<<value<<' '<<match[0]<<' '<<match[1]; // 3 0 1
```

Time: $\mathcal{O}(N^2M)$

20 lines

```
template<class T>
pair<T, vi>weightedMatching(vector<vector<T>>&C) {
  int i=sz(C), m=i?sz(C[0]):0, c, s, r;
  vector<T>dist(m), pot(m);
  vi match(i), rev(m, -1), cols(m), prev(m);
  T d=0, nd, cost=0;
  while (i--) {
    rep(c, 0, m) dist[c]=C[i][c], cols[c]=c, prev[c]=i;
    for (s=0;;) {
      rep(j, s, m) { c=cols[j], nd=dist[c]-pot[c];
        if (j==s||d>nd) d=nd, swap(cols[s], cols[j]); }
      if ((r=rev[c=cols[s++]])==-1) break;
      rep(j, 0, m) if (dist[j]>(nd=C[r][j]-C[r][c]+dist[c]))
        dist[j]=nd, prev[j]=r; }
    cost+=dist[c];
    while (s--) pot[cols[s]]=dist[cols[s]]-d;
    for (; r!=i; swap(c, match[r]))
      r=rev[c]=prev[c]; }
  return {cost, match};
}
```

GeneralMatching.h

Description: Matching for general graphs. Fails with probability N/mod. The returned pairs may appear in any order.

Usage:

```
vec<pii>edges={{0, 1}, {2, 3}};
auto pairs=generalMatching(4, edges);
cout<<pairs.size(); // 2
```

Time: $\mathcal{O}(N^3)$

"../numerical/MatrixInverse-mod.h" 25 lines

```
vector<pii>generalMatching(int N, vector<pii>&ed) {
  vector<vector<ll>>mat(N, vector<ll>(N)), A;
  for (pii pa:ed) { int a=pa.first, b=pa.second, r=rand()%mod;
    mat[a][b]=r, mat[b][a]=(mod-r)%mod; }
  int r=matInv(A=mat), M=2*N-r, fi, fj; assert(r%2==0);
  if (M!=N) do { mat.resize(M, vector<ll>(M));
    rep(i, 0, N) { mat[i].resize(M);
      rep(j, N, M) { int r=rand()%mod;
        mat[i][j]=r, mat[j][i]=(mod-r)%mod; }
    }
  } while (matInv(A=mat)!=M);
  vi has(M, 1); vector<pii>ret;
  rep(it, 0, M/2) { rep(i, 0, M) if (has[i])
      rep(j, i+1, M) if (A[i][j]&&mat[i][j]) {
        fi=i; fj=j; goto done; }
    assert(0); done:
    if (fj<N) ret.emplace_back(fi, fj);
    has[fi]=has[fj]=0;
    rep(sw, 0, 2) { ll a=modpow(A[fi][fj], mod-2);
      rep(i, 0, M) if (has[i]&&A[i][fj]) {
        ll b=A[i][fj]*a%mod;
        rep(j, 0, M) A[i][j]=(A[i][j]-A[fi][j]*b)%mod; }
      swap(fi, fj); }
  }
  return ret; }
```

7.4 DFS algorithms

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.

Usage:

```
vec<vi>g={{1}, {0, 2}, {}};vec<vi>parts;
scc(g, [&](vi&p){parts.push_back(p);});
cout<<ncomps<<' '<<(comp[0]==comp[1]); // 2 1
```

Time: $\mathcal{O}(E+V)$

13 lines

```
vi val, comp, z, cont; int Time, ncomps;
template<class G, class F>int dfs(int j, G&g, F&f) {
  int low=val[j]=++Time, x; z.push_back(j);
  for (auto e:g[j]) if (comp[e]<0)
    low=min(low, val[e]?:dfs(e, g, f));
  if (low==val[j]) {
    do { x=z.back(); z.pop_back();
      comp[x]=ncomps; cont.push_back(x); } while (x!=j);
    f(cont); cont.clear(); ncomps++; }
  return val[j]=low; }
template<class G, class F>void scc(G&g, F f) { int n=sz(g);
  val.assign(n, 0); comp.assign(n, -1); Time=ncomps=0;
  rep(i, 0, n) if (comp[i]<0) dfs(i, g, f); }
```

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected graph, and runs a callback for the edges in each. In a biconnected component there are at least two internally disjoint paths between any two nodes (a cycle exists through them). Note that a node can be in several components. An edge which is not in a component is a bridge, i.e., not part of any cycle.

Usage:

```
ed={{{1, 0}, {2, 2}}, {{0, 0}, {2, 1}}, {{1, 1}, {0, 2}}};
Time=0;st.clear();vec<vi>blocks;
bicomps([&](vi e){blocks.push_back(e);});
cout<<blocks[0].size(); // 3 edges
```

Time: $\mathcal{O}(E+V)$

18 lines

```
vi num, st; vector<vector<pii>>ed; int Time; template<class F>
int dfs(int at, int par, F&f) { int me=num[at]=++Time, top=me;
  for (auto [y, e]:ed[at]) if (e!=par) {
    if (num[y]) { top=min(top, num[y]);
      if (num[y]<me)
        st.push_back(e);
    } else { int si=sz(st);
      int up=dfs(y, e, f); top=min(top, up);
      if (up==me) { st.push_back(e);
        f(vi(st.begin()+si, st.end())); st.resize(si); }
      else if (up<me) st.push_back(e);
      else { /* e is a bridge */ }
    }
  }
  return top; }
template<class F>
void bicomps(F f) { num.assign(sz(ed), 0);
  rep(i, 0, sz(ed)) if (!num[i]) dfs(i, -1, f); }
```

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||!b)\&\&...$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).

Usage:

```
TwoSat sat(2);sat.either(0, 1);sat.setValue(~0);
if(sat.solve())cout<<sat.values[0]<<' '<<sat.values[1];
// 0 1
```

Time: $\mathcal{O}(N+E)$, where N is the number of boolean variables, and E is the number of clauses.

30 lines

```
struct TwoSat { int N; vector<vi>gr;
  vi values; // 0 = false, 1 = true
  TwoSat(int n=0):N(n), gr(2*n) {}
  int addVar() { // (optional)
    gr.emplace_back(); gr.emplace_back(); return N++; }
  void either(int f, int j) { f=max(2*f, -1-2*f);
    j=max(2*j, -1-2*j); gr[f].push_back(j^1);
    gr[j].push_back(f^1); }
  void setValue(int x) { either(x, x); }
  void atMostOne(const vi&li) { // (optional)
    if (sz(li)<=1) return;
    int cur=~li[0];
    rep(i, 2, sz(li)) { int next=addVar();
      either(cur, ~li[i]); either(cur, next);
      either(~li[i], next); cur=~next; }
    either(cur, ~li[1]); }
  vi val, comp, z; int time=0;
  int dfs(int i) { int low=val[i]=++time, x; z.push_back(i);
    for(int e:gr[i]) if (!comp[e])
      low=min(low, val[e]?:dfs(e));
    if (low==val[i]) do { x=z.back(); z.pop_back();
      comp[x]=low;
      if (values[x>>1]==-1)
        values[x>>1]=x&1; } while (x!=i); return val[i]=low; }
  bool solve() { values.assign(N, -1);
    val.assign(2*N, 0); comp=val;
    rep(i, 0, 2*N) if (!comp[i]) dfs(i);
    rep(i, 0, N) if (comp[2*i]==comp[2*i+1]) return 0;
    return 1; }
};
```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.

Usage:

```
vec<vec<pii>>g={{{1, 0}}, {{0, 0}}};
auto walk=eulerWalk(g, 1, 0);
for(int v:walk)cout<<v; // 01
```

Time: $\mathcal{O}(V+E)$

13 lines

```
vi eulerWalk(vector<vector<pii>>&gr, int nedges, int src=0) {
  int n=sz(gr);
  vi D(n), its(n), eu(nedges), ret, s={src};
  D[src]++; // to allow Euler paths, not just cycles
  while (!s.empty()) {
    int x=s.back(), y, e, &it=its[x], end=sz(gr[x]);
    if (it==end){ ret.push_back(x); s.pop_back(); continue; }
    tie(y, e)=gr[x][it++];
    if (!eu[e]) { D[x]--, D[y]++;
      eu[e]=1; s.push_back(y); }}
  for (int x:D) if (x<0||sz(ret)!=nedges+1) return {};
  return {ret.rbegin(), ret.rend()};
}
```

7.5 Coloring

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D, computes a $(D+1)$-coloring of the edges such that no neighboring edges share a color. (D-coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)

Usage:

```
auto colors=edgeColoring(3, {{0, 1}, {1, 2}, {2, 0}});
cout<<(colors[0]!=colors[1]); // 1
```

Time: $\mathcal{O}(NM)$

22 lines

```
vi edgeColoring(int N, vector<pii>eds) {
  vi cc(N+1), ret(sz(eds)), fan(N), free(N), loc;
  for (pii e:eds)++cc[e.first], ++cc[e.second];
  int u, v, ncols=*max_element(all(cc))+1;
  vector<vi>adj(N, vi(ncols, -1));
  for (pii e:eds) { tie(u, v)=e;
    fan[0]=v; loc.assign(ncols, 0);
    int at=u, end=u, d, c=free[u], ind=0, i=0;
    while (d=free[v], !loc[d]&&(v=adj[u][d])!=-1)
      loc[d]=++ind, cc[ind]=d, fan[ind]=v; cc[loc[d]]=c;
    for (int cd=d; at!=-1; cd^=c^d, at=adj[at][cd])
      swap(adj[at][cd], adj[end=at][cd^c^d]);
    while (adj[fan[i]][d]!=-1) {
      int left=fan[i], right=fan[++i], e=cc[i];
      adj[u][e]=left; adj[left][e]=u; adj[right][e]=-1;
      free[right]=e; }
    adj[u][d]=fan[i]; adj[fan[i]][d]=u;
    for (int y:{fan[0], u, end})
      for (int&z=free[y]=0; adj[y][z]!=-1; z++); }
  rep(i, 0, sz(eds))
    for (tie(u, v)=eds[i]; adj[u][ret[i]]!=v;)++ret[i];
  return ret; }
```

7.6 Heuristics

MaximalCliques.h

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.

Usage:

```
vec<B>g(3);g[0][1]=g[1][0]=1;B P;
rep(i, 0, 3)P[i]=1;int count=0;
cliques(g, [&](B mask){++count;}, P);
cout<<count; // 2:  {0,1} and {2}
```

Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

8 lines

```
typedef bitset<128>B; template<class F>
void cliques(vector<B>&eds, F f, B P=~B(), B X={}, B R={}) {
  if (!P.any()) { if (!X.any()) f(R); return; }
  auto q=(P|X)._Find_first(); auto cands=P&~eds[q];
  rep(i, 0, sz(eds)) if (cands[i]) { R[i]=1;
    cliques(eds, f, P&eds[i], X&eds[i], R);
    R[i]=P[i]=0; X[i]=1; }
}
```

MaximumClique.h

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.

Usage:

```
vb g(3);g[0][1]=g[1][0]=1;
Maxclique solver(g);vi answer=solver.maxClique();
cout<<answer.size(); // 2
```

Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs.

36 lines

```
typedef vector<bitset<200>>vb;
struct Maxclique { double limit=0.025, pk=0;
  struct Vertex { int i, d=0; };
  typedef vector<Vertex>vv; vb e; vv V; vector<vi>C;
  vi qmax, q, S, old;
  void init(vv&r) {
    for (auto&v:r) v.d=0;
    for (auto&v:r) for (auto j:r) v.d+=e[v.i][j.i];
    sort(all(r), [](auto a, auto b) { return a.d>b.d; });
    int mxD=r[0].d; rep(i, 0, sz(r)) r[i].d=min(i, mxD)+1; }
```

```
  void expand(vv&R, int lev=1) { S[lev]+=S[lev-1]-old[lev];
    old[lev]=S[lev-1];
    while (sz(R)) {
      if (sz(q)+R.back().d<=sz(qmax)) return;
      q.push_back(R.back().i); vv T;
      for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
      if (sz(T)) {
        if (S[lev]++/++pk<limit) init(T);
        int j=0, mxk=1, mnk=max(sz(qmax)-sz(q)+1, 1);
        C[1].clear(), C[2].clear();
        for (auto v:T) { int k=1;
          auto f=[&](int i) { return e[v.i][i]; };
          while (any_of(all(C[k]), f)) k++;
          if (k>mxk) mxk=k, C[mxk+1].clear();
          if (k<mnk) T[j++].i=v.i;
          C[k].push_back(v.i); }
        if (j>0) T[j-1].d=0;
        rep(k, mnk, mxk+1) for (int i:C[k])
          T[j].i=i, T[j++].d=k; expand(T, lev+1);
      } else if (sz(q)>sz(qmax)) qmax=q;
      q.pop_back(), R.pop_back(); }
  }
  vi maxClique() { init(V), expand(V); return qmax; }
  Maxclique(vb conn):e(conn), C(sz(e)+1), S(sz(C)), old(S) {
    rep(i, 0, sz(e)) V.push_back({i}); }
};
```

MaximumIndependentSet.h

Description: To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see cover() in Dinic.
Usage:

```
vb g(3), complement(3);g[0][1]=g[1][0]=1;
rep(i, 0, 3)rep(j, 0, 3)
complement[i][j]=(i!=j&&!g[i][j]);
Maxclique solver(complement);auto answer=solver.maxClique();
cout<<answer.size(); // 2
```

7.7 Trees

BinaryLifting.h

Description: One-based binary-lifting LCA for a tree. Set MAXN greater than n and choose LOGN with $2^{LOGN} \geq n$. Vertex 0 is the null ancestor. Add both directions to mp, then run DFS and preprocessing. Clear mp between test cases. Use HLD if memory is tight.
Usage:

```
mp[1]={2,3};mp[2]={1,4};mp[3]={1};mp[4]={2};
dfs(1,0,1);precompute(4);
cout<<get_lca(4,3)<<' '<<get_lca(4,2); // 1 2
```

Memory: $\mathcal{O}(MAXN \cdot LOGN)$.
Time: $\mathcal{O}(N \log N)$ preprocessing, $\mathcal{O}(\log N)$ per query.

14 lines

```
const int MAXN=200005,LOGN=20; vector<int>mp[MAXN];
int dep[MAXN],st[MAXN][LOGN+1];
void dfs(int u,int p,int d){ dep[u]=d;st[u][0]=p;
  for(int v:mp[u])if(v!=p)dfs(v,u,d+1); }
void precompute(int n){
  for(int j=1;j<=LOGN;j++)for(int i=1;i<=n;i++)
    st[i][j]=st[st[i][j-1]][j-1]; }
int get_lca(int u,int v){
  if(dep[u]<dep[v])swap(u,v);
  for(int i=LOGN;i>=0;i--)
    if(dep[u]-(1<<i)>=dep[v])u=st[u][i];
  if(u==v)return u;
  for(int i=LOGN;i>=0;i--)if(st[u][i]!=st[v][i])
    u=st[u][i],v=st[v][i]; return st[u][0]; }
```

LCA.h

Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.
Usage:

```
vec<vi>g={{1, 2}, {0, 3}, {0}, {1}};LCA tree(g);
cout<<tree.lca(3, 2); // 0
```

Time: $\mathcal{O}(N \log N + Q)$

"../data-structures/RMQ.h" 13 lines

```
struct LCA { int T=0; vi time, path, ret; RMQ<int>rmq;
  LCA(vector<vi>&C):time(sz(C)), rmq((dfs(C, 0, -1), ret)) {}
  void dfs(vector<vi>&C, int v, int par) { time[v]=T++;
    for (int y:C[v]) if (y!=par) {
      path.push_back(v), ret.push_back(time[v]); dfs(C, y, v);
    }
  }
  int lca(int a, int b) {
    if (a==b) return a;
    tie(a, b)=minmax(time[a], time[b]);
    return path[rmq.query(a, b)]; }
  //dist(a, b){return depth[a]+depth[b]-2*depth[lca(a, b)];}
};
```

CompressTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S|-1$) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.
Usage:

```
vec<vi>g={{1, 2}, {0}, {0}};LCA lca(g);
auto small=compressTree(lca, vi{1, 2});
for(auto [parent, id]:small)cout<<parent<<':'<<id<<' ';
// 0:0 0:1 0:2
```

Time: $\mathcal{O}(|S| \log |S|)$

"LCA.h" 14 lines

```
typedef vector<pair<int, int>>vpi;
vpi compressTree(LCA&lca, const vi&subset) {
  static vi rev; rev.resize(sz(lca.time));
  vi li=subset, &T=lca.time;
  auto cmp=[&](int a, int b) { return T[a]<T[b]; };
  sort(all(li), cmp); int m=sz(li)-1;
  rep(i, 0, m) { int a=li[i], b=li[i+1];
    li.push_back(lca.lca(a, b)); }
  sort(all(li), cmp); li.erase(unique(all(li)), li.end());
  rep(i, 0, sz(li)) rev[li[i]]=i;
  vpi ret={pii(0, li[0])};
  rep(i, 0, sz(li)-1) { int a=li[i], b=li[i+1];
    ret.emplace_back(rev[lca.lca(a, b)], b); }
  return ret; }
```

HLDLazyAdapter.h

Description: Adapter between the original HLD and the Info/Tag lazy segment tree. Preserves initial values of -1e9 and the original add/max interface.
Usage:

```
HLDCompat::Node st(0, 3);st.add(0, 3, 1000000000);
st.add(1, 3, 5);cout<<st.query(0, 3); // 5
```

Time: $\mathcal{O}(N)$ build and space; $\mathcal{O}(\log N)$ add/query.

"../data-structures/LazySegmentTree.h" 6 lines

```
namespace HLDCompat { struct Node { int lo;
 LazySegmentTree<Info, Tag>st;
 Node(int l, int r):lo(l), st(vec<Info>(r-l, Info(-1000000000))
      ){}
 void add(int l, int r, int x){ st.modify(l-lo, r-lo, Tag{x});
      }
 int query(int l, int r){ return l==r?-1000000000:int(st.query(
      l-lo, r-lo).max_val); }
}; }
```

HLD.h

Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most log(n) light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS_EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0.
Usage:

```
vec<vi>g={{1}, {0, 2}, {1}};HLD<false>hld(g);
hld.modifyPath(0, 2, 1000000000); // initialize to zero
hld.modifyPath(1, 2, 5);
cout<<hld.queryPath(0, 2)<<' '<<hld.querySubtree(1); // 5 5
hld.tree->add(hld.pos[1], hld.pos[1]+hld.siz[1], 2);
cout<<hld.querySubtree(1); // 7, subtree add above
vi rev(3);rep(v, 0, 3)rev[hld.pos[v]]=v;
cout<<rev[hld.pos[2]]; // 2, inverse DFS order
```

Time: $\mathcal{O}\left((\log N)^2\right)$

"HLDLazyAdapter.h" 30 lines

```
template<bool VALS_EDGES>struct HLD { int N, tim=0;
  vector<vi>adj; vi par, siz, rt, pos; HLDCompat::Node*tree;
  HLD(vector<vi>adj_)
    :N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
      rt(N), pos(N), tree(new HLDCompat::Node(0, N)){ dfsSz(0);
            dfsHld(0); }
  void dfsSz(int v) {
    for (int&u:adj[v]) { adj[u].erase(find(all(adj[u]), v));
      par[u]=v; dfsSz(u); siz[v]+=siz[u];
      if (siz[u]>siz[adj[v][0]]) swap(u, adj[v][0]); }
  }
  void dfsHld(int v) { pos[v]=tim++;
    for (int u:adj[v]) { rt[u]=(u==adj[v][0]?rt[v]:u);
      dfsHld(u); }
  }
  template<class B>void process(int u, int v, B op) {
    for (;; v=par[rt[v]]) {
      if (pos[u]>pos[v]) swap(u, v);
      if (rt[u]==rt[v]) break;
      op(pos[rt[v]], pos[v]+1); }
    op(pos[u]+VALS_EDGES, pos[v]+1); }
  void modifyPath(int u, int v, int val) {
    process(u, v, [&](int l, int r) { tree->add(l, r, val); });
  }
  int queryPath(int u, int v) { // Modify depending on problem
    int res=-1e9;
    process(u, v, [&](int l, int r) {
        res=max(res, tree->query(l, r)); }); return res; }
  int querySubtree(int v) { // modifySubtree is similar
    return tree->query(pos[v]+VALS_EDGES, pos[v]+siz[v]); }
};
```

DSUOnTree.h

Description: DSU on tree; ans[u] counts distinct values in u's subtree. Vertices are 1-based, with 0 meaning no parent or heavy child. Set MAXN greater than n and compress values into $[0, MAXN)$. dfs1 resets heavy children. Use keep=false at the root to clear counts after answering; otherwise reset color and diff before reuse. Clear mp between cases. This is an independent snippet; allow enough stack for recursive DFS.
Usage:

```
mp[1]={2,3};mp[2]={1};mp[3]={1};
val[1]=val[2]=1;val[3]=2;
dfs1(1,0);dfs_dsu(1,0,false);
cout<<ans[1]<<' '<<ans[2]; // 2 1
```

Memory: $\mathcal{O}(MAXN)$.
Time: $\mathcal{O}(N \log N)$.

16 lines

```
const int MAXN=200005; vector<int>mp[MAXN];
vector<int>sz=vector<int>(MAXN),val(MAXN),hc(MAXN);
vector<int>ans(MAXN),color(MAXN);int diff=0;
```

```
void dfs1(int u,int fa){ sz[u]=1;hc[u]=0;
  for(int v:mp[u])if(v!=fa){ dfs1(v,u);sz[u]+=sz[v];
    if(!hc[u]||sz[v]>sz[hc[u]])hc[u]=v; }
}
void update(int u,int fa,int flag,int skip){
  if(flag==1){if(++color[val[u]]==1)diff++;}
  else if(--color[val[u]]==0)diff--;
  for(int v:mp[u])if(v!=fa&&v!=skip)update(v,u,flag,skip); }
void dfs_dsu(int u,int fa,bool keep){
  for(int v:mp[u])if(v!=fa&&v!=hc[u])dfs_dsu(v,u,false);
  if(hc[u])dfs_dsu(hc[u],u,true);
  update(u,fa,1,hc[u]);ans[u]=diff;
  if(!keep)update(u,fa,-1,0); }
```

LinkCutTree.h

Description: Represents a forest of unrooted trees. You can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree.

Usage:

```
LinkCut forest(3);forest.link(0, 1);forest.link(1, 2);
cout<<forest.connected(0, 2); // 1
forest.cut(1, 2);cout<<forest.connected(0, 2); // 0
```

Time: All operations take amortized $\mathcal{O}(\log N)$.

58 lines

```
struct Node { // Splay tree. Root's pp contains tree's parent.
  Node*p=0, *pp=0, *c[2]; bool flip=0;
  Node() { c[0]=c[1]=0; fix(); }
  void fix() {
    if (c[0]) c[0]->p=this;
    if (c[1]) c[1]->p=this;
    // (+ update sum of subtree elements etc. if wanted)
  }
  void pushFlip() {
    if (!flip) return;
    flip=0; swap(c[0], c[1]);
    if (c[0]) c[0]->flip^=1;
    if (c[1]) c[1]->flip^=1; }
  int up() { return p?p->c[1]==this:-1; }
  void rot(int i, int b) { int h=i^b;
    Node*x=c[i], *y=b==2?x:x->c[h], *z=b?y:x;
    if ((y->p=p)) p->c[up()]=y;
    c[i]=z->c[i^1];
    if (b<2) { x->c[h]=y->c[h^1];
      y->c[h^1]=x; }
    z->c[i^1]=this; fix(); x->fix(); y->fix();
    if (p) p->fix();
    swap(pp, y->pp); }
  void splay() {
    for (pushFlip(); p; ) {
      if (p->p) p->p->pushFlip();
      p->pushFlip(); pushFlip(); int c1=up(), c2=p->up();
      if (c2==-1) p->rot(c1, 2);
      else p->p->rot(c2, c1!=c2); }
  }
  Node*first() { pushFlip();
    return c[0]?c[0]->first():(splay(), this); }
};
struct LinkCut { vector<Node>node;
  LinkCut(int N):node(N) {}
  void link(int u, int v) { // add an edge (u, v)
    assert(!connected(u, v)); makeRoot(&node[u]);
    node[u].pp=&node[v]; }
  void cut(int u, int v) { // remove an edge (u, v)
    Node*x=&node[u], *top=&node[v]; makeRoot(top); x->splay();
    assert(top==(x->pp?:x->c[0]));
    if (x->pp) x->pp=0;
    else { x->c[0]=top->p=0;
      x->fix(); }
  }
  bool connected(int u, int v) { // are u, v in the same tree?
    Node*nu=access(&node[u])->first();
    return nu==access(&node[v])->first(); }
  void makeRoot(Node*u) { access(u); u->splay();
    if(u->c[0]) { u->c[0]->p=0;
      u->c[0]->flip^=1; u->c[0]->pp=u; u->c[0]=0; u->fix(); }
  }
  Node*access(Node*u) { u->splay();
    while (Node*pp=u->pp) { pp->splay(); u->pp=0;
      if (pp->c[1]) { pp->c[1]->p=0; pp->c[1]->pp=pp; }
      pp->c[1]=u; pp->fix(); u=pp; }
    return u; }
};
```

CentroidDecomposition.h

Description: Recursively find the centroid of a tree and its subtrees. Add answer or parent bookkeeping at the marked callback.

Usage:

```
adj={{1}, {0, 2}, {1}};
is_removed.assign(3, false);subtree_size.resize(3);
int c=get_centroid(0, get_subtree_size(0));cout<<c; // 1
build_centroid_decomp();
```

Time: $\mathcal{O}(N \log N)$

23 lines

```
vector<vector<int>>adj; vector<bool>is_removed;
vector<int>subtree_size;
int get_subtree_size(int node, int parent=-1) {
  subtree_size[node]=1;
  for (int child:adj[node]) {
    if (child==parent||is_removed[child]) { continue; }
    subtree_size[node]+=get_subtree_size(child, node); }
  return subtree_size[node]; }
int get_centroid(int node, int tree_size, int parent=-1) {
  for (int child:adj[node]) {
    if (child==parent||is_removed[child]) { continue; }
    if (subtree_size[child]*2>tree_size) {
      return get_centroid(child, tree_size, node); }
  }
  return node; }
void build_centroid_decomp(int node=0) {
  int centroid=get_centroid(node, get_subtree_size(node));
  // <- do something
  is_removed[centroid]=true;
  for (int child:adj[centroid]) {
    if (is_removed[child]) { continue; }
    build_centroid_decomp(child); }
}
```

7.8 Math

7.8.1 Number of Spanning Trees

Create an $N \times N$ matrix `mat`, and for each edge $a \to b \in G$, do `mat[a][b]--, mat[b][b]++` (and `mat[b][a]--, mat[a][a]++` if G is undirected). Remove the ith row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

7.8.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \cdots \geq d_n$ exists iff $d_1 + \cdots + d_n$ is even and for every $k = 1 \ldots n$,

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

Geometry (8)

8.1 Geometric primitives

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

Usage:

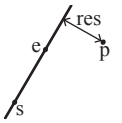
```
Point<ll>a(1, 2), b(3, 4);auto c=a+b;
cout<<c.x<<' '<<c.y<<' '<<a.dot(b); // 4 6 11
```

26 lines

```
template<class T>int sgn(T x) { return (x>0)-(x<0); }
template<class T>
struct Point { typedef Point P; T x, y;
  explicit Point(T x=0, T y=0):x(x), y(y) {}
  bool operator<(P p) const { return tie(x, y)<tie(p.x, p.y); }
  bool operator==(P p) const { return tie(x, y)==tie(p.x, p.y);
       }
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); }
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x+y*p.y; }
  T cross(P p) const { return x*p.y-y*p.x; }
  T cross(P a, P b) const { return (a-*this).cross(b-*this); }
  T dist2() const { return x*x+y*y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return*this/dist(); } // makes dist()=1
  P perp() const { return P(-y, x); } // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
    return P(x*cos(a)-y*sin(a), x*sin(a)+y*cos(a)); }
  friend ostream&operator<<(ostream&os, P p) {
    return os<<"("<<p.x<<","<<p.y<<")"; }
};
```

lineDistance.h

Description:
Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.



Usage:

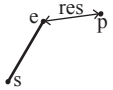
```
using P=Point<double>;
cout<<lineDist(P(0, 0), P(2, 0), P(1, 3)); // 3
```

"Point.h"

3 lines

```
template<class P>
double lineDist(P a, P b, P p) {
  return (double)(b-a).cross(p-a)/(b-a).dist(); }
```

SegmentDistance.h



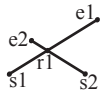
Description:
Returns the shortest distance between point p and the line segment from point s to e.
Usage: `cout<<segDist(P(0, 0), P(2, 0), P(3, 1)); // sqrt(2)`

"Point.h" 5 lines

```
typedef Point<double>P;
double segDist(P s, P e, P p) {
  if (s==e) return (p-s).dist();
  auto d=(e-s).dist2(), t=min(d, max(.0, (p-s).dot(e-s)));
  return ((p-s)*d-(e-s)*t).dist()/d; }
```

SegmentIntersection.h

Description:
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.



Usage:

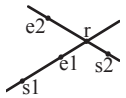
```
using P=Point<double>;
auto pts=segInter(P(0, 0), P(2, 2), P(0, 2), P(2, 0));
if(!pts.empty())cout<<pts[0]; // (1,1)
```

"Point.h", "OnSegment.h" 13 lines

```
template<class P>vector<P>segInter(P a, P b, P c, P d) {
  auto oa=c.cross(d, a), ob=c.cross(d, b),
       oc=a.cross(b, c), od=a.cross(b, d);
  // Checks if intersection is single non-endpoint point.
  if (sgn(oa)*sgn(ob)<0&&sgn(oc)*sgn(od)<0)
    return {(a*ob-b*oa)/(ob-oa)};
  set<P>s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
}
```

lineIntersection.h

Description:
If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.



Usage:

```
using P=Point<double>;
auto [status, p]=lineInter(P(0, 0), P(2, 2), P(0, 2), P(2, 0))
  ;
if(status==1)cout<<p; // (1,1)
```

"Point.h" 8 lines

```
template<class P>
pair<int, P>lineInter(P s1, P e1, P s2, P e2) {
  auto d=(e1-s1).cross(e2-s2);
  if (d==0) // if parallel
    return {-(s1.cross(e1, s2)==0), P(0, 0)};
  auto p=s2.cross(e1, e2), q=s2.cross(e2, s1);
  return {1, (s1*p+e1*q)/d};
}
```

sideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 $\Leftrightarrow$ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.
Usage:

```
using P=Point<ll>;
cout<<sideOf(P(0, 0), P(2, 0), P(1, 1)); // 1
```

"Point.h" 6 lines

```
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }
template<class P>
int sideOf(P s, P e, P p, double eps) {
  auto a=(e-s).cross(p-s); double l=(e-s).dist()*eps;
  return (a>l)-(a<-l); }
```

OnSegment.h

Description: Returns true iff p lies on the line segment from s to e. Use `(segDist(s,e,p)<=epsilon)` instead when using Point<double>.
Usage:

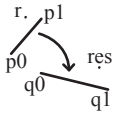
```
using P=Point<ll>;
cout<<onSegment(P(0, 0), P(2, 0), P(1, 0)); // 1
```

"Point.h" 2 lines

```
template<class P>bool onSegment(P s, P e, P p) {
  return p.cross(s, e)==0&&(s-p).dot(e-p)<=0; }
```

linearTransformation.h

Description:



Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

Usage:

```
P out=linearTransformation(P(0, 0), P(1, 0),
P(2, 3), P(4, 3), P(0, 1));cout<<out; // (2,5)
```

"Point.h" 5 lines

```
typedef Point<double>P;
P linearTransformation(P p0, P p1, P q0, P q1, P r) {
  P dp=p1-p0, dq=q1-q0, num(dp.cross(dq), dp.dot(dq));
  return q0+P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
}
```

LineProjectionReflection.h

Description: Projects point p onto line ab. Set refl=true to get reflection of point p across line ab instead. The wrong point will be returned if P is an integer point and the desired point doesn't have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow.
Usage:

```
using P=Point<double>;P a(0, 0), b(2, 0), p(1, 3);
cout<<lineProj(a, b, p); // (1,0)
cout<<lineProj(a, b, p, true); // (1,-3)
```

"Point.h" 3 lines

```
template<class P>
P lineProj(P a, P b, P p, bool refl=false) { P v=b-a;
  return p-v.perp()*(1+refl)*v.cross(p-a)/v.dist2(); }
```

Angle.h

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.
Usage:

```
Angle a(1, 0), b(0, 1);cout<<(a<b); // 1
Angle c=a.t90();cout<<c.x<<' '<<c.y; // 0 1
```

26 lines

```
struct Angle { int x, y; int t;
  Angle(int x, int y, int t=0):x(x), y(y), t(t) {}
  Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }
  int half() const { assert(x||y); return y<0||(y==0&&x<0); }
  Angle t90() const { return {-y, x, t+(half()&&x>=0)}; }
  Angle t180() const { return {-x, -y, t+half()}; }
  Angle t360() const { return {x, y, t+1}; }
};
bool operator<(Angle a, Angle b) {
  // add a.dist2() and b.dist2() to also compare distances
  return make_tuple(a.t, a.half(), a.y*(ll)b.x)<
         make_tuple(b.t, b.half(), a.x*(ll)b.y); }
// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.
pair<Angle, Angle>segmentAngles(Angle a, Angle b) {
  if (b<a) swap(a, b);
  return (b<a.t180()?
          make_pair(a, b):make_pair(b, a.t360())); }
Angle operator+(Angle a, Angle b) { // point a + vector b
  Angle r(a.x+b.x, a.y+b.y, a.t);
  if (a.t180()<r) r.t--;
  return r.t180()<a?r.t360():r; }
Angle angleDiff(Angle a, Angle b) { // angle b - angle a
  int tu=b.t-a.t; a.t=b.t;
  return {a.x*b.x+a.y*b.y, a.x*b.y-a.y*b.x, tu-(b<a)};
}
```

8.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.
Usage:

```
pair<P, P>pts;
if(circleInter(P(0, 0), P(2, 0), 1, 1, &pts))
cout<<pts.first; // (1,0), tangent point repeated
```

"Point.h" 9 lines

```
typedef Point<double>P;
bool circleInter(P a, P b, double r1, double r2, pair<P, P>*out
    ) {
  if (a==b) { assert(r1!=r2); return false; }
  P vec=b-a; double d2=vec.dist2(), sum=r1+r2, dif=r1-r2,
       p=(d2+r1*r1-r2*r2)/(d2*2), h2=r1*r1-p*p*d2;
  if (sum*sum<d2||dif*dif>d2) return false;
  P mid=a+vec*p, per=vec.perp()*sqrt(fmax(0, h2)/d2);
  *out={mid+per, mid-per};
  return true; }
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.
Usage:

```
using P=Point<double>;
auto lines=tangents(P(0, 0), 1, P(4, 0), 1);
cout<<lines.size(); // 2 external tangents
cout<<lines[0].first<<' '<<lines[0].second;
```

"Point.h" 10 lines

```
template<class P>
vector<pair<P, P>>tangents(P c1, double r1, P c2, double r2) {
  P d=c2-c1; double dr=r1-r2, d2=d.dist2(), h2=d2-dr*dr;
  if (d2==0||h2<0)  return {};
  vector<pair<P, P>>out;
  for (double sign:{-1, 1}) {
    P v=(d*dr+d.perp()*sqrt(h2)*sign)/d2;
```

```
    out.push_back({c1+v*r1, c2+v*r2}); }
  if (h2==0) out.pop_back();
  return out; }
```

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. P is intended to be Point<double>.

Usage:

```
using P=Point<double>;
auto pts=circleLine(P(0, 0), 1, P(-2, 0), P(2, 0));
for(P p:pts)cout<<p; // (-1,0)(1,0)
```

"Point.h" 9 lines

```
template<class P>
vec<P>circleLine(P c, double r, P a, P b) {
  P ab=b-a, p=a+ab*(c-a).dot(ab)/ab.dist2();
  double s=a.cross(b, c), h2=r*r-s*s/ab.dist2();
  if (h2<0) return {};
  if (h2==0) return {p};
  P h=ab.unit()*sqrt(h2);
  return {p-h, p+h};
}
```

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon.

Usage:

```
vec<P>p={P(0, 0), P(2, 0), P(2, 2), P(0, 2)};
cout<<circlePoly(P(1, 1), 10, p); // 4
```

Time: $\mathcal{O}(n)$

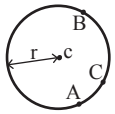
"../../content/geometry/Point.h" 13 lines

```
typedef Point<double>P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P>ps) {
  auto tri=[&](P p, P q) { auto r2=r*r/2; P d=q-p;
    auto a=d.dot(p)/d.dist2(), b=(p.dist2()-r*r)/d.dist2();
    auto det=a*a-b;
    if (det<=0) return arg(p, q)*r2;
    auto s=max(0., -a-sqrt(det)), t=min(1., -a+sqrt(det));
    if (t<0||1<=s) return arg(p, q)*r2;
    P u=p+d*s, v=q+d*(t-1);
    return arg(p, u)*r2+u.cross(v)/2+arg(v, q)*r2; };
  auto sum=0.0; rep(i, 0, sz(ps))
    sum+=tri(ps[i]-c, ps[(i+1)%sz(ps)]-c); return sum; }
```

circumcircle.h

Description:

The circumcirle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.



Usage:

```
P a(0, 0), b(2, 0), c(0, 2);
cout<<ccCenter(a, b, c)<<' '<<ccRadius(a, b, c);
// (1,1), approximately 1.41421
```

"Point.h" 6 lines

```
typedef Point<double>P;
double ccRadius(P A, P B, P C) {
  return (B-A).dist()*(C-B).dist()*(A-C).dist()/
      abs((B-A).cross(C-A))/2; }
P ccCenter(P A, P B, P C) { P b=C-A, c=B-A;
  return A+(b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2; }
```

GeometryExtras.h

Description: perpen returns two points on the perpendicular bisector of distinct a,b. incircle returns center/radius of a nondegenerate triangle. circularSegmentArea uses radius r and central angle theta in [0,2*pi].

Usage:

```
auto [a, b]=perpen(P(0, 0), P(2, 0));cout<<a; // (1,0)
auto [c, r]=incircle(P(0, 0), P(4, 0), P(0, 3));
cout<<c<<' '<<r; // (1,1) 1
cout<<circularSegmentArea(2, acos(-1.0)); // 2*pi
```

Time: $\mathcal{O}(1)$.

"Point.h" 12 lines

```
using P=Point<double>;
pair<P, P>perpen(P a, P b) {
 P M=(a+b)/2, c=b-M, d(-c.y, c.x);return {M, M+d};
}
pair<P, double>incircle(P A, P B, P C) {
 double a=(B-C).dist(), b=(A-C).dist(), c=(A-B).dist();
 P center=(A*a+B*b+C*c)/(a+b+c);
 double r=abs((B-A).cross(C-A))/(a+b+c);
 return {center, r};
}
double circularSegmentArea(double r, double theta) {
 return 0.5*r*r*(theta-sin(theta)); }
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.

Usage:

```
vec<P>p={P(0, 0), P(2, 0)};
auto [center, radius]=mec(p);
cout<<center<<' '<<radius; // (1,0) 1
```

Time: expected $\mathcal{O}(n)$

"circumcircle.h" 13 lines

```
pair<P, double>mec(vector<P>ps) {
  shuffle(all(ps), mt19937(time(0))); P o=ps[0];
  double r=0, EPS=1+1e-8;
  rep(i, 0, sz(ps)) if ((o-ps[i]).dist()>r*EPS) {
    o=ps[i], r=0;
    rep(j, 0, i) if ((o-ps[j]).dist()>r*EPS) {
      o=(ps[i]+ps[j])/2; r=(o-ps[i]).dist();
      rep(k, 0, j) if ((o-ps[k]).dist()>r*EPS) {
        o=ccCenter(ps[i], ps[j], ps[k]); r=(o-ps[i]).dist(); }
    }
  }
  return {o, r};
}
```

8.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage:

```
vec<P>p={P(0, 0), P(2, 0), P(2, 2), P(0, 2)};
cout<<inPolygon(p, P(1, 1)); // 1
cout<<inPolygon(p, P(0, 1), false); // 1
```

Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h" 8 lines

```
template<class P>
bool inPolygon(vector<P>&p, P a, bool strict=true) {
  int cnt=0, n=sz(p);
  rep(i, 0, n) { P q=p[(i+1)%n];
    if (onSegment(p[i], q, a)) return!strict;
    //or: if (segDist(p[i], q, a) <= eps) return !strict;
    cnt^=((a.y<p[i].y)-(a.y<q.y))*a.cross(p[i], q)>0; }
  return cnt; }
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

Usage:

```
using P=Point<ll>;
vec<P>p={P(0, 0), P(2, 0), P(0, 2)};
cout<<polygonArea2(p); // 4; area is 2
```

"Point.h" 3 lines

```
template<class T>
T polygonArea2(vector<Point<T>>&v) { T a=v.back().cross(v[0]);
  rep(i, 0, sz(v)-1) a+=v[i].cross(v[i+1]); return a; }
```

PolygonCenter.h

Description: Returns the center of mass for a polygon.

Usage:

```
vec<P>p={P(0, 0), P(2, 0), P(2, 2), P(0, 2)};
P center=polygonCenter(p);cout<<center; // (1,1)
```

Time: $\mathcal{O}(n)$

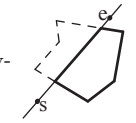
"Point.h" 6 lines

```
typedef Point<double>P;
P polygonCenter(const vector<P>&v) { P res(0, 0); double A=0;
  for (int i=0, j=sz(v)-1; i<sz(v); j=i++) {
    res=res+(v[i]+v[j])*v[j].cross(v[i]); A+=v[j].cross(v[i]);
  }
  return res/A/3; }
```

PolygonCut.h

Description:

Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.



Usage:

```
vec<P>p={P(0, 0), P(2, 0), P(2, 2), P(0, 2)};
auto kept=polygonCut(p, P(1, -1), P(1, 3));
cout<<kept.size(); // 4, right half of the square
```

"Point.h" 11 lines

```
typedef Point<double>P;
vector<P>polygonCut(const vector<P>&poly, P s, P e) {
  vector<P>res;
  rep(i, 0, sz(poly)) {
    P cur=poly[i], prev=i?poly[i-1]:poly.back();
    auto a=s.cross(e, cur), b=s.cross(e, prev);
    if ((a<0)!=(b<0))
      res.push_back(cur+(prev-cur)*(a/(a-b)));
    if (a<0)
      res.push_back(cur); }
  return res; }
```

HalfPlaneIntersection.h

Description: Intersects the left half-planes of directed lines using the original snippet. Uses its own Point/Vector helpers inside namespace SnippetHPI. Assumes a nonempty, well-conditioned system forming a bounded nondegenerate polygon; empty, unbounded and degenerate classification is not provided. Mutates line order.

Usage:

```
using namespace SnippetHPI;
vec<HLine>h={{Point(0, 0), Point(1, 0)},
{Point(1, 0), Point(1, 1)}, {Point(1, 1), Point(0, 1)},
{Point(0, 1), Point(0, 0)}};
auto polygon=halfPlaneIntersection(h);
cout<<polygon.size(); // 4
```

Time: $\mathcal{O}(N \log N)$ time, $\mathcal{O}(N)$ space.

57 lines

```
namespace SnippetHPI { const double eps=1e-8;
int sgn(double x) { if (fabs(x)<eps) return 0;
  if (x<0) return-1; return 1; }
  struct Point { double x, y;
  Point(double _x=0, double _y=0):x(_x), y(_y) {}
```

```
  Point operator+(const Point&b) const { return Point(x+b.x, y+
      b.y); }
  Point operator-(const Point&b) const { return Point(x-b.x, y-
      b.y); }
  Point operator*(const double&k) const { return Point(x*k, y*k
      ); }
  Point operator/(const double&k) const { return Point(x/k, y/k
      ); }
  bool operator==(const Point&b) const {
    return sgn(x-b.x)==0&&sgn(y-b.y)==0; } };
typedef Point Vector; double dot(Vector a, Vector b) {
  return a.x*b.x+a.y*b.y; }
double cross(Vector a, Vector b) { return a.x*b.y-a.y*b.x; }
  double length(Vector a) { return sqrt(dot(a, a)); }
  double dist(Point a, Point b) { return length(a-b); }
  bool isUp(Vector a) {
  return sgn(a.y)>0||(sgn(a.y)==0&&sgn(a.x)>=0); }
bool polarCmp(Point a, Point b) {
  bool upA=isUp(a), upB=isUp(b);
  if (upA!=upB) return upA>upB;
  return sgn(cross(a, b))>0; }
  struct HLine { Point s, e;
  double angle; HLine() {}
  HLine(Point _s, Point _e):s(_s), e(_e) {
    angle=atan2(e.y-s.y, e.x-s.x); } };
bool isPointOnRight(Point p, HLine L) {
  return sgn(cross(L.e-L.s, p-L.s))<0; }
bool cmpHLine(const HLine&a, const HLine&b) {
  int d=sgn(a.angle-b.angle); if (d!=0) return d<0;
  return sgn(cross(a.e-a.s, b.e-a.s))>0; }
Point getIntersect(HLine l1, HLine l2) {
  Vector u=l1.s-l2.s, v=l1.e-l1.s, w=l2.e-l2.s;
  double t=cross(w, u)/cross(v, w); return l1.s+v*t; }
vec<Point>halfPlaneIntersection(vec<HLine>&lines) {
  int n=lines.size();
  sort(lines.begin(), lines.end(), cmpHLine); int m=1;
  for (int i=1; i<n; i++) {
    if (sgn(lines[i].angle-lines[i-1].angle)!=0) {
      lines[m++]=lines[i]; } }
      n=m; deque<HLine>q;
  deque<Point>pts; q.push_back(lines[0]);
  for (int i=1; i<n; i++) {
    while (pts.size()>0&&isPointOnRight(pts.back(), lines[i]))
        {
      q.pop_back(); pts.pop_back(); }
    while (pts.size()>0&&isPointOnRight(pts.front(), lines[i]))
        {
      q.pop_front(); pts.pop_front(); }
    if (sgn(cross(q.back().e-q.back().s, lines[i].e-lines[i].s)
        )==0) continue;
    pts.push_back(getIntersect(q.back(), lines[i]));
    q.push_back(lines[i]); }
  while (pts.size()>0&&isPointOnRight(pts.back(), q.front())) {
    q.pop_back(); pts.pop_back(); }
  while (pts.size()>0&&isPointOnRight(pts.front(), q.back())) {
    q.pop_front(); pts.pop_front(); }
    if (q.size()>1) {
    pts.push_back(getIntersect(q.back(), q.front())); }
  return vec<Point>(pts.begin(), pts.end()); } }
```

ConvexHull.h

Description:
Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.



Usage:

```
vec<P>p={P(0, 0), P(2, 0), P(0, 2), P(0, 0)};
auto hull=convexHull(p);cout<<hull.size(); // 3
```

Time: $\mathcal{O}(n \log n)$

"Point.h" 10 lines

```
typedef Point<ll>P;
vector<P>convexHull(vector<P>pts) {
  if (sz(pts)<=1) return pts;
  sort(all(pts)); vector<P>h(sz(pts)+1); int s=0, t=0;
  for (int it=2; it--; s=--t, reverse(all(pts)))
    for (P p:pts) {
      while (t>=s+2&&h[t-2].cross(h[t-1], p)<=0) t--;
      h[t++]=p; }
  return {h.begin(), h.begin()+t-(t==2&&h[0]==h[1])};
}
```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).

Usage:

```
vec<P>h={P(0, 0), P(2, 0), P(2, 2), P(0, 2)};
auto pair=hullDiameter(h);
cout<<(pair[0]-pair[1]).dist2(); // 8
```

Time: $\mathcal{O}(n)$

"Point.h" 8 lines

```
typedef Point<ll>P;
array<P, 2>hullDiameter(vector<P>S) { int n=sz(S), j=n<2?0:1;
  pair<ll, array<P, 2>>res({0, {S[0], S[0]}}); rep(i, 0, j)
    for (;; j=(j+1)%n) {
      res=max(res, {(S[i]-S[j]).dist2(), {S[i], S[j]}});
      if ((S[(j+1)%n]-S[j]).cross(S[i+1]-S[i])>=0)
        break; }
  return res.second; }
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.

Usage:

```
vec<P>h={P(0, 0), P(2, 0), P(2, 2), P(0, 2)};
cout<<inHull(h, P(1, 1)); // 1
cout<<inHull(h, P(0, 1), false); // 1
```

Time: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h" 10 lines

```
typedef Point<ll>P;
bool inHull(const vector<P>&l, P p, bool strict=true) {
  int a=1, b=sz(l)-1, r=!strict;
  if (sz(l)<3) return r&&onSegment(l[0], l.back(), p);
  if (sideOf(l[0], l[a], l[b])>0) swap(a, b);
  if (sideOf(l[0], l[a], p)>=r||sideOf(l[0], l[b], p)<=-r)
    return false;
  while (abs(a-b)>1) { int c=(a+b)/2;
    (sideOf(l[0], l[c], p)>0?b:a)=c; }
  return sgn(l[a].cross(l[b], p))<r; }
```

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $\bullet$ $(-1,-1)$ if no collision, $\bullet$ $(i,-1)$ if touching the corner i, $\bullet$ (i,i) if along side $(i,i+1)$, $\bullet$ (i,j) if crossing sides $(i,i+1)$ and $(j,j+1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i,i+1)$. The points are returned in the same order as the line hits the polygon. `extrVertex` returns the point of a hull with the max projection onto a line.

Usage:

```
using P=Point<ll>;
vec<P>h={P(0, 0), P(2, 0), P(2, 2), P(0, 2)};
auto hit=lineHull(P(-1, 1), P(3, 1), h);
cout<<hit[0]<<' '<<hit[1]; // intersected edge indices
```

Time: $\mathcal{O}(\log n)$

"Point.h" 29 lines

```
#define cmp(i, j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n])
    )
#define extr(i) cmp(i+1, i)>=0&&cmp(i, i-1+n)<0
template<class P>int extrVertex(vector<P>&poly, P dir) {
  int n=sz(poly), lo=0, hi=n;
  if (extr(0)) return 0;
  while (lo+1<hi) { int m=(lo+hi)/2;
    if (extr(m)) return m;
    int ls=cmp(lo+1, lo), ms=cmp(m+1, m);
    (ls<ms||(ls==ms&&ls==cmp(lo, m))?hi:lo)=m; }
  return lo; }
#define cmpL(i) sgn(a.cross(poly[i], b))
template<class P>
array<int, 2>lineHull(P a, P b, vector<P>&poly) {
  int endA=extrVertex(poly, (a-b).perp());
  int endB=extrVertex(poly, (b-a).perp());
  if (cmpL(endA)<0||cmpL(endB)>0)
    return {-1, -1};
  array<int, 2>res;
  rep(i, 0, 2) { int lo=endB, hi=endA, n=sz(poly);
    while ((lo+1)%n!=hi) { int m=((lo+hi+(lo<hi?0:n))/2)%n;
      (cmpL(m)==cmpL(endB)?lo:hi)=m; }
    res[i]=(lo+!cmpL(hi))%n; swap(endA, endB); }
  if (res[0]==res[1]) return {res[0], -1};
  if (!cmpL(res[0])&&!cmpL(res[1]))
    switch ((res[0]-res[1]+sz(poly)+1)%sz(poly)) {
      case 0:return {res[0], res[0]};
      case 2:return {res[1], res[1]};
    }
  return res; }
```

8.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.

Usage:

```
auto [a, b]=closest(vec<P>{P(0, 0), P(1, 0), P(9, 9)});
cout<<(a-b).dist2(); // 1
```

Time: $\mathcal{O}(n \log n)$

"Point.h" 13 lines

```
typedef Point<ll>P;
pair<P, P>closest(vector<P>v) { assert(sz(v)>1); set<P>S;
  sort(all(v), [](P a, P b) { return a.y<b.y; });
  pair<ll, pair<P, P>>ret{LLONG_MAX, {P(), P()}};
  int j=0;
  for (P p:v) {
    P d{1+(ll)sqrt(ret.first), 0};
    while (v[j].y<=p.y-d.x) S.erase(v[j++]);
    auto lo=S.lower_bound(p-d), hi=S.upper_bound(p+d);
    for (; lo!=hi;++lo)
      ret=min(ret, {(*lo-p).dist2(), {*lo, p}}); S.insert(p);
  }
  return ret.second; }
```

kdTree.h

Description: KD-tree (2d, can be extended to 3d)

Usage:

```
vec<P>p={P(0, 0), P(5, 0)};KDTree tree(p);
auto [d, q]=tree.nearest(P(1, 0));
cout<<d<<' '<<q; // 1 (0,0)
```

"Point.h" 43 lines

```
typedef long long T; typedef Point<T>P;
const T INF=numeric_limits<T>::max();
bool on_x(P a, P b) { return a.x<b.x; }
bool on_y(P a, P b) { return a.y<b.y; }
```

```
struct Node {
  P pt; // if this is a leaf, the single point in it
  T x0=INF, x1=-INF, y0=INF, y1=-INF; // bounds
  Node*first=0, *second=0;
  T distance(P p) { // min squared distance to a point
    T x=(p.x<x0?x0:p.x>x1?x1:p.x);
    T y=(p.y<y0?y0:p.y>y1?y1:p.y); return (P(x, y)-p).dist2();
  }
  Node(vector<P>&&vp):pt(vp[0]) {
    for (P p:vp) { x0=min(x0, p.x); x1=max(x1, p.x);
      y0=min(y0, p.y); y1=max(y1, p.y); }
    if (vp.size()>1) {
      // split on x if width >= height (not ideal...)
      sort(all(vp), x1-x0>=y1-y0?on_x:on_y);
      // divide by taking half the array for each child (not
      // best performance with many duplicates in the middle)
      int half=sz(vp)/2;
      first=new Node({vp.begin(), vp.begin()+half});
      second=new Node({vp.begin()+half, vp.end()}); }
  }
};
struct KDTree { Node*root;
  KDTree(const vector<P>&vp):root(new Node({all(vp)})) {}
  pair<T, P>search(Node*node, P p) {
    if (!node->first) {
      // uncomment if we should not find the point itself:
      // if (p==node->pt) return {INF, P()};
      return make_pair((p-node->pt).dist2(), node->pt); }
    Node*f=node->first, *s=node->second;
    T bfirst=f->distance(p), bsec=s->distance(p);
    if (bfirst>bsec) swap(bsec, bfirst), swap(f, s);
    // search closest side first, other side if needed
    auto best=search(f, p);
    if (bsec<best.first)
      best=min(best, search(s, p)); return best; }
  // find nearest point to a point, and its squared distance
  // (requires an arbitrary operator< for Point)
  pair<T, P>nearest(P p) { return search(root, p); }
};
```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ...}, all counter-clockwise.

Usage:

```
vec<P>p={P(0, 0), P(2, 0), P(0, 2)};
auto triangles=triangulate(p);
cout<<triangles.size()/3; // 1
```

Time: $\mathcal{O}(n \log n)$

"Point.h" 67 lines

```
typedef Point<ll>P; typedef struct Quad*Q;
typedef __int128_t lll; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX, LLONG_MAX); // not equal to any other point
struct Quad { Q rot, o; P p=arb; bool mark;
  P&F() { return r()->p; }
  Q&r() { return rot->rot; }
  Q prev() { return rot->o->rot; }
  Q next() { return r()->prev(); }
}*H;
bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
  lll p2=p.dist2(), A=a.dist2()-p2,
      B=b.dist2()-p2, C=c.dist2()-p2;
  return p.cross(a, b)*C+p.cross(b, c)*A+p.cross(c, a)*B>0; }
Q makeEdge(P orig, P dest) {
  Q r=H?H:new Quad{new Quad{new Quad{new Quad{0}}}};
  H=r->o; r->r()->r()=r;
  rep(i, 0, 4) r=r->rot, r->p=arb, r->o=i&1?r:r->r();
  r->p=orig; r->F()=dest; return r; }
void splice(Q a, Q b) {
  swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o); }
Q connect(Q a, Q b) { Q q=makeEdge(a->F(), b->p);
  splice(q, a->next()); splice(q->r(), b); return q; }
pair<Q, Q>rec(const vector<P>&s) {
  if (sz(s)<=3) {
    Q a=makeEdge(s[0], s[1]), b=makeEdge(s[1], s.back());
    if (sz(s)==2) return { a, a->r() };
    splice(a->r(), b); auto side=s[0].cross(s[1], s[2]);
    Q c=side?connect(b, a):0;
    return {side<0?c->r():a, side<0?c:b->r() };
  }
#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base))>0)
  Q A, B, ra, rb; int half=sz(s)/2;
  tie(ra, A)=rec({all(s)-half});
  tie(B, rb)=rec({sz(s)-half+all(s)});
  while ((B->p.cross(H(A))<0&&(A=A->next()))||
         (A->p.cross(H(B))>0&&(B=B->r()->o)));
  Q base=connect(B->r(), A);
  if (A->p==ra->p) ra=base->r();
  if (B->p==rb->p) rb=base;
#define DEL(e, init, dir) Q e=init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
      Q t=e->dir; \
      splice(e, e->prev()); \
      splice(e->r(), e->r()->prev()); \
      e->o=H; H=e; e=t; \
    }
  for (;;) { DEL(LC, base->r(), o);  DEL(RC, base, prev());
    if (!valid(LC)&&!valid(RC)) break;
    if (!valid(LC)||(valid(RC)&&circ(H(RC), H(LC))))
      base=connect(RC, base->r());
    else
      base=connect(base->r(), LC->r()); }
  return { ra, rb };
}
vector<P>triangulate(vector<P>pts) {
  sort(all(pts));  assert(unique(all(pts))==pts.end());
  if (sz(pts)<2) return {};
  Q e=rec(pts).first;
  vector<Q>q={e};
  int qi=0;
  while (e->o->F().cross(e->F(), e->p)<0) e=e->o;
#define ADD { Q c=e; do { c->mark=1; pts.push_back(c->p); \
  q.push_back(c->r()); c=c->next(); } while (c!=e); }
  ADD; pts.clear();
  while (qi<sz(q)) if (!(e=q[qi++])->mark) ADD;
  return pts; }
```

8.5 3D

PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

Usage:

```
using P=Point3D<double>;struct Face{int a, b, c;};
vec<P>p={P(0, 0, 0), P(1, 0, 0), P(0, 1, 0), P(0, 0, 1)};
vec<Face>f={{0, 2, 1}, {0, 1, 3}, {0, 3, 2}, {1, 2, 3}};
cout<<signedPolyVolume(p, f); // 1/6
```

5 lines

```
template<class V, class L>
double signedPolyVolume(const V&p, const L&trilist) {
  double v=0;
  for (auto i:trilist) v+=p[i.a].cross(p[i.b]).dot(p[i.c]);
  return v/6; }
```

Point3D.h

Description: Class to handle points in 3D space. T can be e.g. double or long long.

Usage:

```
Point3D<double>a(1, 0, 0), b(0, 1, 0);
auto c=a.cross(b);cout<<c.x<<' '<<c.y<<' '<<c.z;
// 0 0 1
```

28 lines

```
template<class T>struct Point3D { typedef Point3D P;
  T x, y, z;
  explicit Point3D(T x=0, T y=0, T z=0):x(x), y(y), z(z) {}
  bool operator<(P p) const {
    return tie(x, y, z)<tie(p.x, p.y, p.z); }
  bool operator==(P p) const {
    return tie(x, y, z)==tie(p.x, p.y, p.z); }
  P operator+(P p) const { return P(x+p.x, y+p.y, z+p.z); }
  P operator-(P p) const { return P(x-p.x, y-p.y, z-p.z); }
  P operator*(T d) const { return P(x*d, y*d, z*d); }
  P operator/(T d) const { return P(x/d, y/d, z/d); }
  T dot(P p) const { return x*p.x+y*p.y+z*p.z; }
  P cross(P p) const {
    return P(y*p.z-z*p.y, z*p.x-x*p.z, x*p.y-y*p.x); }
  T dist2() const { return x*x+y*y+z*z; }
  double dist() const { return sqrt((double)dist2()); }
  //Azimuthal angle (longitude) to x-axis in interval [-pi, pi]
  double phi() const { return atan2(y, x); }
  //Zenith angle (latitude) to the z-axis in interval [0, pi]
  double theta() const { return atan2(sqrt(x*x+y*y), z); }
  P unit() const { return*this/(T)dist(); } //makes dist()=1
  //returns unit vector normal to *this and p
  P normal(P p) const { return cross(p).unit(); }
  //returns point rotated 'angle' radians ccw around axis
  P rotate(double angle, P axis) const {
    double s=sin(angle), c=cos(angle); P u=axis.unit();
    return u*dot(u)*(1-c)+(*this)*c-cross(u)*s; }
};
```

3dHull.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.

Usage:

```
vec<P3>p={P3(0, 0, 0), P3(1, 0, 0), P3(0, 1, 0), P3(0, 0, 1)};
auto faces=hull3d(p);cout<<faces.size(); // 4
for(auto f:faces)cout<<f.a<<' '<<f.b<<' '<<f.c<<' ';
```

Time: $\mathcal{O}(n^2)$

"Point3D.h" 34 lines

```
typedef Point3D<double>P3;
struct PR {
  void ins(int x) { (a==-1?a:b)=x; }
  void rem(int x) { (a==x?a:b)=-1; }
  int cnt() { return (a!=-1)+(b!=-1); }
  int a, b; };
struct F { P3 q; int a, b, c; };
vector<F>hull3d(const vector<P3>&A) { assert(sz(A)>=4);
  vector<vector<PR>>E(sz(A), vector<PR>(sz(A), {-1, -1}));
#define E(x, y) E[f.x][f.y]
  vector<F>FS;
  auto mf=[&](int i, int j, int k, int l) {
    P3 q=(A[j]-A[i]).cross((A[k]-A[i]));
    if (q.dot(A[l])>q.dot(A[i]))
      q=q*-1;
    F f{q, i, j, k};
    E(a, b).ins(k); E(a, c).ins(j); E(b, c).ins(i);
    FS.push_back(f); };
  rep(i, 0, 4) rep(j, i+1, 4) rep(k, j+1, 4)
    mf(i, j, k, 6-i-j-k);
  rep(i, 4, sz(A)) {
    rep(j, 0, sz(FS)) { F f=FS[j];
```

```
      if(f.q.dot(A[i])>f.q.dot(A[f.a])) { E(a, b).rem(f.c);
        E(a, c).rem(f.b); E(b, c).rem(f.a);
        swap(FS[j--], FS.back()); FS.pop_back(); }
    }
    int nw=sz(FS);
    rep(j, 0, nw) { F f=FS[j];
#define C(a, b, c) if (E(a, b).cnt()!=2) mf(f.a, f.b, i, f.c);
      C(a, b, c); C(a, c, b); C(b, c, a); }
  }
  for (F&it:FS) if ((A[it.b]-A[it.a]).cross(
    A[it.c]-A[it.a]).dot(it.q)<=0) swap(it.c, it.b);
  return FS; };
```

sphericalDistance.h

Description: Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (latitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points.

Usage:

```
double pi=acos(-1.0);
cout<<sphericalDistance(0, pi/2, pi/2, pi/2, 1);
// pi/2
```

6 lines

```
double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius) {
  double dx=sin(t2)*cos(f2)-sin(t1)*cos(f1);
  double dy=sin(t2)*sin(f2)-sin(t1)*sin(f1);
  double dz=cos(t2)-cos(t1); double d=sqrt(dx*dx+dy*dy+dz*dz);
  return radius*2*asin(d/2); }
```

Strings (9)

KMP.h

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.

Usage:

```
auto ans=match("ababa", "aba");
for(int i:ans)cout<<i<<' '; // 0 2
```

Time: $\mathcal{O}(n)$

9 lines

```
vi pi(const string&s) { vi p(sz(s));
  rep(i, 1, sz(s)) { int g=p[i-1];
    while (g&&s[i]!=s[g]) g=p[g-1];
    p[i]=g+(s[i]==s[g]); }
  return p; }
vi match(const string&s, const string&pat) {
  vi p=pi(pat+'\0'+s), res; rep(i, sz(p)-sz(s), sz(p))
    if (p[i]==sz(pat)) res.push_back(i-2*sz(pat));
  return res; }
```

Zfunc.h

Description: z[i] is the LCP of s[i:] and s, except z[0]=0. For extended KMP, run Z on pattern + separator + text; choose a separator absent from both. Then z[pattern.size()+1+i] is the LCP of the pattern and the text suffix at i.

Usage:

```
string pat="aba", s="ababa";auto z=Z(pat+'#'+s);
rep(i, 0, sz(s))cout<<z[sz(pat)+1+i]<<' '; // 3 0 3 0 1
```

Time: $\mathcal{O}(n)$

7 lines

```
vi Z(const string&S) { vi z(sz(S)); int l=-1, r=-1;
  rep(i, 1, sz(S)) { z[i]=i>=r?0:min(r-i, z[i-l]);
    while (i+z[i]<sz(S)&&S[i+z[i]]==S[z[i]])
      z[i]++;
    if (i+z[i]>r)
      l=i, r=i+z[i]; }
  return z; }
```

Manacher.h

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).

Usage:

```
string s="abba";auto p=manacher(s);int start=0, len=0;
rep(z, 0, 2)rep(i, 0, sz(p[z])) {
int k=2*p[z][i]+z;
if(k>len)len=k, start=i-p[z][i];
}
cout<<s.substr(start, len); // abba
```

Time: $\mathcal{O}(N)$

10 lines

```
array<vi, 2>manacher(const string&s) { int n=sz(s);
  array<vi, 2>p={vi(n+1), vi(n)};
  rep(z, 0, 2) for (int i=0, l=0, r=0; i<n; i++) {
    int t=r-i+!z;
    if (i<r) p[z][i]=min(t, p[z][l+t]);
    int L=i-p[z][i], R=i+p[z][i]-!z;
    while (L>=1&&R+1<n&&s[L-1]==s[R+1])
      p[z][i]++, L--, R++;
    if (R>r) l=L, r=R; }
  return p; }
```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.

Usage:

```
string s="baca";int k=minRotation(s);
rotate(s.begin(), s.begin()+k, s.end());
cout<<s; // abac
```

Time: $\mathcal{O}(N)$

6 lines

```
int minRotation(string s) { int a=0, N=sz(s); s+=s;
  rep(b, 0, N) rep(k, 0, N) {
    if (a+k==b||s[a+k]<s[b+k]) {b+=max(0, k-1); break;}
    if (s[a+k]>s[b+k]) { a=b; break; }
  }
  return a; }
```

SuffixArray.h

Description: Builds suffix array for a string. sa[i] is the starting index of the suffix which is i'th in the sorted suffix array. The returned vector is of size $n+1$, and sa[0] = n. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: lcp[i] = lcp(sa[i], sa[i-1]), lcp[0] = 0. The input string must not contain any nul chars.

Usage:

```
SuffixArray sa("aba");
for(int i:sa.sa)cout<<i<<' '; // 3 2 0 1
cout<<sa.lcp[2]; // 1
```

Time: $\mathcal{O}(n \log n)$

18 lines

```
struct SuffixArray { vi sa, lcp;
  SuffixArray(string s, int lim=256) { // or vector<int>
    s.push_back(0); int n=sz(s), k=0, a, b;
    vi x(all(s)), y(n), ws(max(n, lim));
    sa=lcp=y, iota(all(sa), 0);
    for (int j=0, p=0; p<n; j=max(1, j*2), lim=p) {
      p=j, iota(all(y), n-j);
      rep(i, 0, n) if (sa[i]>=j) y[p++]=sa[i]-j;
      fill(all(ws), 0); rep(i, 0, n) ws[x[i]]++;
      rep(i, 1, lim) ws[i]+=ws[i-1];
      for (int i=n; i--;) sa[--ws[x[y[i]]]]=y[i];
      swap(x, y), p=1, x[sa[0]]=0;
      rep(i, 1, n) a=sa[i-1], b=sa[i], x[b]=
        (y[a]==y[b]&&y[a+j]==y[b+j])?p-1:p++; }
    for (int i=0, j; i<n-1; lcp[x[i++]]=k)
      for (k&&k--, j=sa[x[i]-1];
          s[i+k]==s[j+k]; k++); }
};
```

SuffixTree.h

Description: Ukkonen's algorithm for online suffix tree construction. Each node contains indices [l, r) into the string, and a list of child nodes. Suffixes are given by traversals of this tree, joining [l, r) substrings. The root is 0 (has l = -1, r = 0), non-existent children are -1. To get a complete tree, append a dummy symbol – otherwise it may contain an incomplete path (still useful for substring matching, though). Allocate the large object on the heap. The LCS helper additionally requires ALPHA=28.

Usage:

```
auto st=make_unique<SuffixTree>("aba");
int child=st->t[0]['a'-'a'];
if(child!=-1)cout<<st->a.substr(st->l[child],
st->r[child]-st->l[child]); // root edge label
```

Time: $\mathcal{O}(26N)$

38 lines

```
struct SuffixTree {
  enum { N=200010, ALPHA=26 }; // N ~ 2*maxlen+10
  int toi(char c) { return c-'a'; }
  string a; // v = cur node, q = cur position
  int t[N][ALPHA], l[N], r[N], p[N], s[N], v=0, q=0, m=2;
  void ukkadd(int i, int c) { suff:
    if (r[v]<=q) {
      if (t[v][c]==-1) { t[v][c]=m;  l[m]=i;
        p[m++]=v; v=s[v]; q=r[v];  goto suff; }
      v=t[v][c]; q=l[v]; }
    if (q==-1||c==toi(a[q])) q++; else {
      l[m+1]=i;  p[m+1]=m;  l[m]=l[v];  r[m]=q;
      p[m]=p[v];  t[m][c]=m+1;  t[m][toi(a[q])]=v;
      l[v]=q;  p[v]=m;  t[p[m]][toi(a[l[m]])]=m;
      v=s[p[m]];  q=l[m];
      while (q<r[m]) { v=t[v][toi(a[q])];  q+=r[v]-l[v]; }
      if (q==r[m])  s[m]=v;  else s[m]=m+2;
      q=r[v]-(q-r[m]);  m+=2;  goto suff; }
  }
  SuffixTree(string a):a(a) { fill(r, r+N, sz(a));
    memset(s, 0, sizeof s); memset(t, -1, sizeof t);
    fill(t[1], t[1]+ALPHA, 0);
    s[0]=1; l[0]=l[1]=-1; r[0]=r[1]=p[0]=p[1]=0;
    rep(i, 0, sz(a)) ukkadd(i, toi(a[i])); }
  // example: find longest common substring (uses ALPHA = 28)
  pii best;
  int lcs(int node, int i1, int i2, int olen) {
    if (l[node]<=i1&&i1<r[node]) return 1;
    if (l[node]<=i2&&i2<r[node]) return 2;
    int mask=0, len=node?olen+(r[node]-l[node]):0;
    rep(c, 0, ALPHA) if (t[node][c]!=-1)
      mask|=lcs(t[node][c], i1, i2, len);
    if (mask==3)
      best=max(best, {len, r[node]-len}); return mask; }
  static pii LCS(string s, string t) {
    SuffixTree st(s+(char)('z'+1)+t+(char)('z'+2));
    st.lcs(0, sz(s), sz(s)+1+sz(t), 0); return st.best; }
};
```

Hashing.h

Description: Self-explanatory methods for string hashing.

Usage:

```
string s="ababa";HashInterval h(s);
cout<<(h.hashInterval(0, 3)==h.hashInterval(2, 5)); // 1
cout<<h.hashInterval(0, 3).get(); // numeric hash
```

35 lines

```
// Arithmetic mod 2^64-1. 2x slower than mod 2^64 and more
// code, but works on evil test data (e.g. Thue-Morse, where
```

```
// ABBA... and BAAB... of length 2^10 hash the same mod 2^64).
// "typedef ull H;" instead if you think test data is random,
// or work mod 10^9+7 if the Birthday paradox is not a problem.
typedef uint64_t ull;
struct H {
  ull x; H(ull x=0):x(x) {}
  H operator+(H o) { return x+o.x+(x+o.x<x); }
  H operator-(H o) { return*this+~o.x; }
  H operator*(H o) { auto m=(__uint128_t)x*o.x;
    return H((ull)m)+(ull)(m>>64); }
  ull get() const { return x+!~x; }
  bool operator==(H o) const { return get()==o.get(); }
  bool operator<(H o) const { return get()<o.get(); }
};
static const H C=(ll)1e11+3; // (order ~ 3e9; random also ok)
struct HashInterval { vector<H>ha, pw;
  HashInterval(string&str):ha(sz(str)+1), pw(ha) { pw[0]=1;
    rep(i, 0, sz(str))
      ha[i+1]=ha[i]*C+str[i],
      pw[i+1]=pw[i]*C; }
  H hashInterval(int a, int b) { // hash [a, b)
    return ha[b]-ha[a]*pw[b-a]; }
};
vector<H>getHashes(string&str, int length) {
  if (sz(str)<length) return {};
  H h=0, pw=1; rep(i, 0, length)
    h=h*C+str[i], pw=pw*C;
  vector<H>ret={h};
  rep(i, length, sz(str)) {
    ret.push_back(h=h*C+str[i]-pw*str[i-length]); }
  return ret; }
H hashString(string&s){H h{};
for(char c:s) h=h*C+c;return h;}
```

AhoCorasick.h

Description: Lowercase AC automaton with completed trie transitions and fail-tree frequency accumulation. Keep ac global; MAXN must exceed the sum of pattern lengths and every pattern ID. Duplicate patterns share a node. Insert all words before init; match counts visits without early exit. Build the fail tree and accumulate once, then cnt[end[i]] is word i's count. clear before reuse; insert every queried ID again. Deep fail trees need enough stack for recursive dfs. Uncomment the alert return for early exit only; it stops counting. init and build_tree must each run once per case.

Usage:

```
ac.insert(1,"a");ac.insert(2,"aa");ac.init();
ac.match("aaa");ac.build_tree();ac.dfs(0);
cout<<ac.cnt[ac.end[1]]<<' '<<ac.cnt[ac.end[2]]; // 3 2
ac.clear(); // next case
```

Memory: $\mathcal{O}(26 \cdot MAXN)$.

Time: $\mathcal{O}(26S + T + P)$ for total pattern length S, text length T, pattern count P.

33 lines

```
struct ACAM {
  static const int MAXN=200005; // > total pattern length and
      pattern IDs
  int trie[MAXN][26]{},fail[MAXN]{},cnt[MAXN]{},alert[MAXN]{},
      end[MAXN]{},tot=0;
  vector<int>mp[MAXN];
  ACAM(){clear();}
  void clear(){
    for(int i=0;i<=tot;i++){
      memset(trie[i],0,sizeof(trie[i]));mp[i].clear();
      fail[i]=alert[i]=cnt[i]=0; }
    tot=0; }
  int insert(int i,const string&s){
    int u=0;for(char c:s){int v=c-'a';
      if(!trie[u][v]){trie[u][v]=++tot;}
      u=trie[u][v]; }
    alert[u]=1;end[i]=u;return u; }
  void init(){
    queue<int>q;for(int i=0;i<26;i++)if(trie[0][i])q.push(trie
        [0][i]);
    while(!q.empty()){
      int u=q.front();q.pop();alert[u]|=alert[fail[u]];
      for(int i=0;i<26;i++)if(trie[u][i]){
        fail[trie[u][i]]=trie[fail[u]][i];q.push(trie[u][i]);
      }else trie[u][i]=trie[fail[u]][i];
    }
  }
  bool match(const string&t){
    int u=0;for(char c:t){u=trie[u][c-'a'];cnt[u]++;
      // if(alert[u])return true; // early exit disables full
          counts
    }
    return false; }
  void build_tree(){for(int i=1;i<=tot;i++)mp[fail[i]].
      push_back(i);}
  void dfs(int u){for(int v:mp[u]){dfs(v);cnt[u]+=cnt[v];}}
}
ac;
```

SAM.h

Description: Suffix automaton for a..z. Keep sam global; MAXN bounds the string length. Build the suffix-link tree and run dfs(0) once to accumulate occurrence counts. dfs returns the maximum of length times occurrences over substrings occurring at least twice. Call init before each case. The separator '{' needs 27 transition columns and both loops changed to 27. Recursive dfs needs enough stack for a deep suffix-link tree.

Usage:

```
string str="ababa";sam.init();for(char c:str)sam.insert(c);
sam.build_tree();cout<<sam.dfs(0); // 6
```

Memory: $\mathcal{O}(26 \cdot MAXN)$, with space for $2 \cdot MAXN$ states.

Time: $\mathcal{O}(26N)$ for a string of length N.

27 lines

```
struct SAM { static const int MAXN=1000007;
  int tot=0,last=0,len[2*MAXN],link[2*MAXN],cnt[2*MAXN];
  int tran[2*MAXN][26];vector<int>mp[2*MAXN];
  void init(){
    for(int i=0;i<=tot;i++){
      mp[i].clear();cnt[i]=len[i]=link[i]=0;
      for(int j=0;j<26;j++)tran[i][j]=0; }
    last=tot=0;link[0]=-1; }
  void insert(char ori){ int c=ori-'a',np=++tot;
    cnt[np]=1;tran[last][c]=np;len[np]=len[last]+1;
    int p=link[last];last=np;
    while(p!=-1&&!tran[p][c]){tran[p][c]=np;p=link[p];}
    if(p==-1){link[np]=0;return;}
    int q=tran[p][c];
    if(len[q]==len[p]+1){link[np]=q;return;}
    int nq=++tot;link[nq]=link[q];
    link[q]=link[np]=nq;len[nq]=len[p]+1;
    for(int i=0;i<26;i++)tran[nq][i]=tran[q][i];
    while(p!=-1&&tran[p][c]==q){tran[p][c]=nq;p=link[p];}
  }
  void build_tree(){for(int i=1;i<=tot;i++)mp[link[i]].
      push_back(i);}
  ll dfs(int i){
    ll maxn=0;for(int v:mp[i]){maxn=max(maxn,dfs(v));cnt[i]+=
        cnt[v];}
    if(cnt[i]>1){maxn=max(maxn,(ll)cnt[i]*len[i]);}
    return maxn; }
}
sam;
```

Trie.h

Description: Lowercase dictionary with duplicate counts. `find` counts exact words; `countPrefix` counts inserted words with the given prefix, including duplicates. The empty prefix counts all inserted words.

Usage:

```
Trie tr;tr.insert("abc");tr.insert("abc");
cout<<tr.find("abc")<<' '<<tr.find("ab"); // 2 0
cout<<tr.countPrefix("ab"); // 2
```

Time: $\mathcal{O}(26N)$

30 lines

```
struct Trie {
    struct TrieNode {
        array<int, 26>nxt={};
        int stop=0, pass=0; };
    int node_count=0; vector<TrieNode>info;
    Trie():info(1) {}
    Trie(int N):info(N) {}
    void insert(const string&word) { int node=0;
        info[node].pass++;
        for(const char&c:word) {
            // If a node with the current char doesn't exist
                create one.
            if(info[node].nxt[c-'a']==0) {
                info[node].nxt[c-'a']=++node_count;
                if(node_count>=sz(info)) {
                    info.emplace_back(); }
            }
            // Move down the path in the trie.
            node=info[node].nxt[c-'a']; info[node].pass++; }
        info[node].stop++; }
    int find(const string&word) { int node=0;
        for(const char&c:word) {
            if(info[node].nxt[c-'a']==0) {return 0;}
            node=info[node].nxt[c-'a']; }
        return info[node].stop; }
    int countPrefix(const string&prefix) { int node=0;
        for(char c:prefix) {
            if(!info[node].nxt[c-'a'])return 0;
```

```
        node=info[node].nxt[c-'a']; }
      return info[node].pass; }
};
```

Utf8Split.h

Description: Splits valid UTF-8 into strings containing one Unicode code point each. Combining marks remain separate; input bytes are not validated.
Usage:

```
auto parts=split_utf8("A\u4E2D\U0001F60A");
cout<<parts.size()<<' '<<parts[1].size(); // 3 3
```

Time: $\mathcal{O}(N)$ time and space for N input bytes.

8 lines

```
vec<string>split_utf8(const string&s) { vec<string>res;
 for(size_t i=0;i<s.length();) {
  int len=1;unsigned char c=s[i];
  if(c>=0xF0)len=4;
  else if(c>=0xE0)len=3;
  else if(c>=0xC0)len=2;
  res.push_back(s.substr(i, len));i+=len; }
 return res; }
```

Various (10)

10.1 Intervals

IntervalContainer.h

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive).
Usage:

```
set<pii>intervals;addInterval(intervals, 1, 4);
addInterval(intervals, 3, 7);removeInterval(intervals, 2, 5);
for(auto [l, r]:intervals)cout<<l<<','<<r<<' ';
// 1,2 5,7
```

Time: $\mathcal{O}(\log N)$

14 lines

```
set<pii>::iterator addInterval(set<pii>&is, int L, int R) {
  if (L==R) return is.end();
  auto it=is.lower_bound({L, R}), before=it;
  while (it!=is.end()&&it->first<=R) { R=max(R, it->second);
    before=it=is.erase(it); }
  if (it!=is.begin()&&(--it)->second>=L) {
    L=min(L, it->first); R=max(R, it->second); is.erase(it); }
  return is.insert(before, {L, R}); }
void removeInterval(set<pii>&is, int L, int R) {
  if (L==R) return;
  auto it=addInterval(is, L, R); auto r2=it->second;
  if (it->first==L) is.erase(it);
  else (int&)it->second=L;
  if (R!=r2) is.emplace(R, r2); }
```

IntervalCover.h

Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add `|| R.empty()`. Returns empty set on failure (or if G is empty).
Usage:

```
vec<pii>intervals={{0, 2}, {1, 4}, {2, 3}};
auto ids=cover(pii{0, 4}, intervals);
for(int i:ids)cout<<i<<' '; // 0 1
```

Time: $\mathcal{O}(N \log N)$

12 lines

```
template<class T>
vi cover(pair<T, T>G, vector<pair<T, T>>I) { vi S(sz(I)), R;
  iota(all(S), 0);
  sort(all(S), [&](int a, int b) { return I[a]<I[b]; });
  T cur=G.first; int at=0;
  while (cur<G.second) { // (A)
    pair<T, int>mx=make_pair(cur, -1);
    while (at<sz(I)&&I[S[at]].first<=cur) {
      mx=max(mx, make_pair(I[S[at]].second, S[at])); at++; }
    if (mx.second==-1) return {};
    cur=mx.first; R.push_back(mx.second); }
  return R; }
```

ConstantIntervals.h

Description: Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback g for each such interval.
Usage:

```
vec<tuple<int, int, int>>runs;
constantIntervals(0, 6, [](int i){return i/2;},
[&](int l, int r, int v){runs.emplace_back(l, r, v);});
for(auto [l, r, v]:runs)cout<<l<<','<<r<<':'<<v<<' ';
// 0,2:0 2,4:1 4,6:2
```

Time: $\mathcal{O}\left(k \log \frac{n}{k}\right)$

14 lines

```
template<class F, class G, class T>
void rec(int from, int to, F&f, G&g, int&i, T&p, T q) {
  if (p==q) return;
  if (from==to) { g(i, to, p);
    i=to; p=q;
  } else { int mid=(from+to)>>1;
    rec(from, mid, f, g, i, p, f(mid));
    rec(mid+1, to, f, g, i, p, q); }
}
template<class F, class G>
void constantIntervals(int from, int to, F f, G g) {
  if (to<=from) return;
  int i=from; auto p=f(i), q=f(to-1);
  rec(from, to-1, f, g, i, p, q); g(i, to, q); }
```

10.2 Misc. algorithms

TernarySearch.h

Description: Find the smallest i in $[a, b]$ that maximizes $f(i)$, assuming that $f(a) < \ldots < f(i) \geq \cdots \geq f(b)$. To reverse which of the sides allows non-strict inequalities, change the $<$ marked with (A) to $<=$, and reverse the loop at (B). To minimize f, change it to $>$, also at (B).
Usage:

```
auto f=[](int x){return -(x-3)*(x-3);};
int pos=ternSearch(0, 10, f);
cout<<pos<<' '<<f(pos); // 3 0
```

Time: $\mathcal{O}(\log(b-a))$

7 lines

```
template<class F>
int ternSearch(int a, int b, F f) { assert(a<=b);
  while (b-a>=5) { int mid=(a+b)/2;
    if (f(mid)<f(mid+1)) a=mid; // (A)
    else b=mid+1; }
  rep(i, a+1, b+1) if (f(a)<f(i)) a=i; // (B)
  return a; }
```

LIS.h

Description: Compute indices for the longest increasing subsequence.
Usage:

```
vec<int>a={3, 1, 2, 5};auto ids=lis(a);
for(int i:ids)cout<<a[i]<<' '; // 1 2 5
```

Time: $\mathcal{O}(N \log N)$

12 lines

```
template<class I>vi lis(const vector<I>&S) {
  if (S.empty()) return {};
  vi prev(sz(S)); typedef pair<I, int>p; vector<p>res;
  rep(i, 0, sz(S)) {
    // change 0 -> i for longest non-decreasing subsequence
    auto it=lower_bound(all(res), p{S[i], 0});
    if (it==res.end()) res.emplace_back(), it=res.end()-1;
    *it={S[i], i};
    prev[i]=it==res.begin()?0:(it-1)->second; }
  int L=sz(res), cur=res.back().second; vi ans(L);
  while (L--) ans[L]=cur, cur=prev[cur];
  return ans; }
```

FastKnapsack.h

Description: Given N non-negative integer weights w and a non-negative target t, computes the maximum S <= t such that S is the sum of some subset of the weights.
Usage: `cout<<knapsack(vi{3, 5, 7}, 10); // 10`
Time: $\mathcal{O}(N \max(w_i))$

10 lines

```
int knapsack(vi w, int t) { int a=0, b=0, x;
  while (b<sz(w)&&a+w[b]<=t) a+=w[b++];
  if (b==sz(w)) return a;
  int m=*max_element(all(w)); vi u, v(2*m, -1); v[a+m-t]=b;
  rep(i, b, sz(w)) { u=v;
    rep(x, 0, m) v[x+w[i]]=max(v[x+w[i]], u[x]);
    for (x=2*m;--x>m;) rep(j, max(0, u[x]), v[x])
      v[x-w[j]]=max(v[x-w[j]], j); }
  for (a=t; v[a+m-t]<0; a--) ;
  return a; }
```

10.3 Dynamic programming

DigitDP.h

Description: Generic digit-DP recipe for nonnegative decimal integers. Fill in the state capacity, initial state, boundary result, transitions, and optional invalid-state pruning for your problem. The state must retain all information needed by future digits. Memoize only non-limited states after the number has started. Each bound resets memo; choose the boundary result to include or exclude zero. Counts must fit in long long.
Usage:

```
// Customize all state placeholders first.
long long L=1,R=100;
cout<<count_valid(R)-count_valid(L-1);
```

Memory: $\mathcal{O}(25S)$.
Time: $\mathcal{O}(10DS)$ for D digits and S states.

19 lines

```
long long memo[25][MAX_STATE];string s;
long long dfs(int pos,int state,bool is_limit,bool is_lead){
  // 边界返回值 (通常是 1 或 !is_lead)
  if(pos==(int)s.length())return boundary_return_val;
  if(!is_limit&&!is_lead&&memo[pos][state]!=-1)return memo[pos
      ][state];
  long long res=0;int up=is_limit?s[pos]-'0':9;
  for(int i=0;i<=up;++i){
    // if (invalid_condition) continue; // 非法条件剪枝
    if(is_lead){
      if(i==0)res+=dfs(pos+1,lead_zero_state,is_limit&&(i==up),
          true);
      else res+=dfs(pos+1,lead_first_digit_state,is_limit&&(i==
          up),false);
    }else res+=dfs(pos+1,next_state,is_limit&&(i==up),false);
  }
  if(!is_limit&&!is_lead)memo[pos][state]=res;
  return res; }
long long count_valid(long long x){
  if(x<0)return 0;
  s=to_string(x);memset(memo,-1,sizeof(memo));
  return dfs(0,initial_state,true,true); }
```

KnuthDP.h

Description: When doing DP on intervals: $a[i][j] = \min_{i<k<j}(a[i][k] + a[k][j]) + f(i,j)$, where the (minimal) optimal k increases with both i and j, one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j-1]$ and $p[i+1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b,c) \le f(a,d)$ and $f(a,c) + f(b,d) \le f(a,d) + f(b,c)$ for all $a \le b \le c \le d$. Consider also: LineContainer (ch. Data structures), monotone queues, ternary search. The example evaluates a small recurrence directly; apply the optimal-split bounds above for the optimization.

Usage:

```
ll dp[3][3]={}, pref[4]={0, 1, 3, 6};
for(int len=2;len<=3;++len)for(int l=0;l+len<=3;++l){
int r=l+len-1;dp[l][r]=LLONG_MAX;
for(int k=l;k<r;++k)dp[l][r]=min(dp[l][r],
dp[l][k]+dp[k+1][r]+pref[r+1]-pref[l]);
}
cout<<dp[0][2]; // 9
```

Time: $\mathcal{O}\left(N^2\right)$

DivideAndConquerDP.h

Description: Given $a[i] = \min_{lo(i)\le k<hi(i)}(f(i,k))$ where the (minimal) optimal k increases with i, computes $a[i]$ for $i = L..R-1$. The example uses $dp[i][k] = (i-k)^2$ for $0 \le k < i$; `res[i]` stores the best split and its cost.

Usage:

```
DP solver;solver.solve(1, 4);
cout<<res[3].first<<' '<<res[3].second; // 2 1
```

Time: $\mathcal{O}\left((N + (hi - lo))\log N\right)$

15 lines

```
struct DP { // Modify at will:
  int lo(int ind) { return 0; }
  int hi(int ind) { return ind; }
  ll f(int ind, int k) { return dp[ind][k]; }
  void store(int ind, int k, ll v) { res[ind]=pii(k, v); }
  void rec(int L, int R, int LO, int HI) {
    if (L>=R) return;
    int mid=(L+R)>>1; pair<ll, int>best(LLONG_MAX, LO);
    rep(k, max(LO, lo(mid)), min(HI, hi(mid)))
      best=min(best, make_pair(f(mid, k), k));
    store(mid, best.second, best.first);
    rec(L, mid, LO, best.second+1);
    rec(mid+1, R, best.second, HI); }
  void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
};
```

10.4 Debugging tricks

- `signal(SIGSEGV, [](int) { _Exit(0); });` converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). `_GLIBCXX_DEBUG` failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).
- `feenableexcept(29);` kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

10.5 Optimization tricks

`__builtin_ia32_ldmxcsr(40896);` disables denormals (which make floats 20x slower near their minimum value).

10.5.1 Pragmas

- `#pragma GCC optimize ("Ofast")` will make GCC auto-vectorize loops and optimizes floating points better.
- `#pragma GCC target ("avx2")` can double performance of vectorized code, but causes crashes on old machines.
- `#pragma GCC optimize ("trapv")` kills the program on integer overflows (but is really slow).

FastMod.h

Description: Compute $a\%b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to a (mod b) in the range $[0, 2b)$.

Usage:

```
FastMod fm(7);auto r=fm.reduce(100);
if(r>=fm.b)r-=fm.b;cout<<r; // 2
```

6 lines

```
typedef unsigned long long ull;
struct FastMod { ull b, m;
  FastMod(ull b):b(b), m(-1ULL/b) {}
  ull reduce(ull a) { // a % b + (0 or b)
    return a-(ull)((__uint128_t(m)*a)>>64)*b; }
};
```

FastInput.h

Description: Read an integer from stdin. Usage requires your program to pipe in input from file.

Usage:

```
int a=readInt(), b=readInt();
cout<<a+b; // stdin:  12 -5 => 7
```

Time: About 5x as fast as cin/scanf.

11 lines

```
inline char gc() { // like getchar()
  static char buf[1<<16]; static size_t bc, be;
  if (bc>=be) { buf[0]=0, bc=0;
    be=fread(buf, 1, sizeof(buf), stdin); }
  return buf[bc++]; // returns 0 on EOF
}
int readInt() { int a, c;
  while ((a=gc())<40);
  if (a=='-') return-readInt();
  while ((c=gc())>=48) a=a*10+c-480;
  return a-48; }
```

BumpAllocator.h

Description: When you need to dynamically allocate many objects and don't care about freeing them. "new X" otherwise has an overhead of something like 0.05us + 16 bytes per allocation.

Usage: `int*p=new int(7);cout<<*p; // 7`

5 lines

```
// Either globally or in a single class:
static char buf[450<<20];
void*operator new(size_t s) { static size_t i=sizeof buf;
  assert(s<i); return (void*)&buf[i-=s]; }
void operator delete(void*) {}
```

SmallPtr.h

Description: A 32-bit pointer that points into BumpAllocator memory.

Usage:

```
ptr<int>p=new int(7);
if(p)cout<<*p; // 7
```

"BumpAllocator.h"

8 lines

```
template<class T>struct ptr { unsigned ind;
  ptr(T*p=0):ind(p?unsigned((char*)p-buf):0) {
    assert(ind<sizeof buf); }
  T&operator*() const { return*(T*)(buf+ind); }
  T*operator->() const { return&**this; }
  T&operator[](int a) const { return (&**this)[a]; }
  explicit operator bool() const { return ind; }
};
```

BumpAllocatorSTL.h

Description: BumpAllocator for STL containers. Keep total allocation below the shared arena size; deallocation does not reclaim space.

Usage:

```
vec<int, small<int>>a;a.push_back(7);
cout<<a[0]; // 7
```

8 lines

```
char buf[450<<20] alignas(16); size_t buf_ind=sizeof buf;
template<class T>struct small { typedef T value_type;
  small() {}
  template<class U>small(const U&) {}
  T*allocate(size_t n) { buf_ind-=n*sizeof(T);
    buf_ind&=0-alignof(T); return (T*)(buf+buf_ind); }
  void deallocate(T*, size_t) {}
};
```

SIMD.h

Description: Cheat sheet of SSE/AVX intrinsics, for doing arithmetic on several numbers at once. Can provide a constant factor improvement of about 4, orthogonal to loop unrolling. Operations follow the pattern `"_mm(256)?_name_(si(128|256)|epi(8|16|32|64)|pd|ps)"`. Not all are described here; grep for `_mm_` in `/usr/lib/gcc/*/4.9/include/` for more. If AVX is unsupported, try 128-bit operations, "emmintrin.h" and `#define __SSE__` and `__MMX__` before including it. For aligned memory use `_mm_malloc(size, 32)` or `int buf[N] alignas(32)`, but prefer loadu/storeu.

Usage:

```
mi v=_mm256_set1_epi32(3);
cout<<sumi32(v); // 24
short a[2]={1, 5}, b[2]={2, 3};
cout<<example_filteredDotProduct(2, a, b); // 2
```

36 lines

```
#pragma GCC target ("avx2") // or sse4.1
#include "immintrin.h"
typedef __m256i mi;
#define L(x) _mm256_loadu_si256((mi*)&(x))
// High-level/specific methods:
// load(u)?_si256, store(u)?_si256, setzero_si256, _mm_malloc
// blendv_(epi8|ps|pd) (z?y:x), movemask_epi8 (hibits of bytes)
// i32gather_epi32(addr, x, 4): map addr[] over 32-b parts of x
// sad_epu8: sum of absolute differences of u8, outputs 4xi64
// maddubs_epi16: dot product of unsigned i7's, outputs 16xi15
// madd_epi16: dot product of signed i16's, outputs 8xi32
// extractf128_si256(, i) (256->128), cvtsi128_si32 (128->lo32)
// permute2f128_si256(x,x,1) swaps 128-bit lanes
// shuffle_epi32(x, 3*64+2*16+1*4+0) == x for each lane
// shuffle_epi8(x, y) takes a vector instead of an imm
// Methods that work with most data types (append e.g. _epi32):
// set1, blend (i8?x:y), add, adds (sat.), mullo, sub, and/or,
// andnot, abs, min, max, sign(1,x), cmp(gt|eq), unpack(lo|hi)
int sumi32(mi m) { union {int v[8]; mi m;}
u; u.m=m; int ret=0; rep(i, 0, 8) ret+=u.v[i]; return ret; }
mi zero() { return _mm256_setzero_si256(); }
mi one() { return _mm256_set1_epi32(-1); }
bool all_zero(mi m) { return _mm256_testz_si256(m, m); }
bool all_one(mi m) { return _mm256_testc_si256(m, one()); }
ll example_filteredDotProduct(int n, short*a, short*b) {
  int i=0; ll r=0; mi zero=_mm256_setzero_si256(), acc=zero;
  while (i+16<=n) { mi va=L(a[i]), vb=L(b[i]); i+=16;
    va=_mm256_and_si256(_mm256_cmpgt_epi16(vb, va), va);
    mi vp=_mm256_madd_epi16(va, vb);
    acc=_mm256_add_epi64(_mm256_unpacklo_epi32(vp, zero),
      _mm256_add_epi64(acc, _mm256_unpackhi_epi32(vp, zero)));
  }
```

```
  union {ll v[4]; mi m;}
  u; u.m=acc; rep(i, 0, 4) r+=u.v[i];
  for (;i<n;++i) if (a[i]<b[i]) r+=a[i]*b[i]; // <- equiv
  return r; }
```

Random.h

Description: Fixed-seed mt19937 and usage fragments. Paste inside a function after declaring v, L, R and mod. Draws use inclusive [L,R]; mod must be positive. The modulo draw can be biased. Use mt19937_64 for 64-bit draws.
Usage:

```
vec<int>v={1, 2, 3};int L=1, R=10, mod=7;
// Paste fragment here.
cout<<rand_val; // in [1,10]
for(int x:v)cout<<x<<' '; // shuffled permutation
```

Time: $\mathcal{O}(N)$ shuffle; expected $\mathcal{O}(1)$ per draw; $\mathcal{O}(1)$ extra space.

3 lines

```
mt19937 rng(810975); shuffle(v.begin(), v.end(), rng);
int rand_val=uniform_int_distribution<int>(L, R)(rng);
int simple_rand=rng()%mod;
```

Timer.h

Description: Runs until close to the time limit. Good for brute forcing solutions. Keep `start` and `timeLimit` outside the loop and refresh `now` inside. Print the saved answer at the marked timeout location before `exit(0)`.
Usage:

```
int best=42;
cout<<best; // 42
```

6 lines

```
auto start=chrono::high_resolution_clock::now();
auto timeLimit=chrono::milliseconds(900); // <- set to close to
     time limit
auto now=chrono::high_resolution_clock::now();
if (now-start>=timeLimit) {
    // <- output answer answer
    exit(0); }
```

Reference (11)

11.1 STL Reference

HashTables.cpp

15 lines

```
unordered_map<int, int>mp; mp.reserve(100005);
// Reserve to reduce rehashing.
// mp[key]=val; // Insert/update.
// mp.count(key); // Test existence: 1 or 0.
// mp.erase(key); // Erase.
// for (auto const& [k, v] : mp) { /* use k and v */ }
unordered_set<int>us; us.reserve(100005);
// Custom hash
struct custom_hash { static uint64_t splitmix64(uint64_t x) {
    x+=0x9e3779b97f4a7c15; x=(x^(x>>30))*0xbf58476d1ce4e5b9;
    x=(x^(x>>27))*0x94d049bb133111eb; return x^(x>>31); }
  size_t operator()(uint64_t x) const {
    static const uint64_t FIXED_RANDOM=chrono::steady_clock::
        now().time_since_epoch().count();
    return splitmix64(x+FIXED_RANDOM); } };
unordered_map<ll, int, custom_hash>safe_map;
```

PriorityQueue.cpp

16 lines

```
priority_queue<int>maxheap;
priority_queue<int, vec<int>, greater<int>>minheap;
auto cmp=[](pair<int, int>a, pair<int, int>b){
 return a.second>b.second; };
priority_queue<pair<int, int>, vec<pair<int, int>>, decltype(
    cmp)>pq(cmp);
struct Type{int property;};
vec<Type>arr;
sort(arr.begin(),arr.end(),[=](const auto&a,const auto&b){
 return a.property<b.property; // ascending
});
auto byProperty=[=](const auto&a,const auto&b){
 return a.property>b.property; // min-heap: larger values have
      lower priority
};
priority_queue<Type,vec<Type>,decltype(byProperty)>propertyHeap
     (byProperty);
// [=] captures outer local variables by value.
// Check empty() before top()/pop().
```

Algorithms.cpp

17 lines

```
// Comparator
bool compare(const std::pair<int, int>&a, const std::pair<int,
     int>&b) {
return a.first<b.first; // Ascending by first.
}
sort(nums.begin(), nums.end());
stable_sort(nums.begin(), nums.end());
nth_element(nums.begin(), nums.begin()+k, nums.end());
// nth_element does not sort the other positions.
binary_search(nums.begin(), nums.end(), x);
next_permutation(nums.begin(), nums.end());
prev_permutation(nums.begin(), nums.end());
accumulate(nums.begin(), nums.end(), 0LL);
iota(nums.begin(), nums.end(), 0);
nums.erase(unique(nums.begin(), nums.end()), nums.end());
// Sort first if removing all duplicate values.
min_element(nums.begin(), nums.end());
max_element(nums.begin(), nums.end());
```

IteratorRules.cpp

6 lines

```
// begin/end form [begin,end); end() is not dereferenceable.
// --begin() and ++end() are invalid.
// vec reallocation invalidates all iterators/references;
// erase invalidates those at/after the erased position.
// map/set insertion preserves iterators; erase invalidates
// only iterators/references to erased elements.
```

11.2 Bitmask Reference

BitmaskReference.cpp

77 lines

```
// Personal bitmask snippet
#define bit(S, i) (((S)>>(i))&1) // Test bit i of S.
#define set_bit(S, i) ((S)|(1<<(i))) // Set bit i.
#define clear_bit(S, i) ((S)&~(1<<(i))) // Clear bit i.
#define toggle_bit(S, i) ((S)^(1<<(i))) // Toggle bit i.
#define lowbit(x) ((x)&-(x)) // Lowest set bit, as in a Fenwick
     tree.
// __builtin_popcount(S):set-bit count; use popcountll for ll.
// __builtin_ctz(S):trailing zeros/lowest set-bit index; S
// != 0.
// Enumerate every nonempty submask of S.
for (int sub=S; sub; sub=(sub-1)&S) {
// sub is a submask of S.
  int remain=S^sub; // The remaining elements of S.
// Example DP transition:
// dp[S]=min(dp[S], dp[sub]+dp[remain]);
  }
// Integer widths: personal macros use int shifts.
// For set/clear/toggle, use 0 <= i < 31 on signed int.
// 64-bit unsigned versions allow 0 <= i < 64.
#define bit64(S, i) (((S)>>(i))&1ULL)
#define set_bit64(S, i) ((S)|(1ULL<<(i)))
#define clear_bit64(S, i) ((S)&~(1ULL<<(i)))
#define toggle_bit64(S, i) ((S)^(1ULL<<(i)))
// lowbit(x) is also valid for uint64_t; zero returns zero.
// Avoid negating the minimum signed integer.
__builtin_popcount(S); // 32-bit; zero is allowed.
__builtin_popcountll(S); // 64-bit; zero is allowed.
__builtin_ctz(S); __builtin_ctzll(S); // S != 0.
31-__builtin_clz(S); // Highest set bit, 32-bit; S != 0.
63-__builtin_clzll(S); // Highest set bit, 64-bit; S != 0.
// Only set-bit positions: O(popcount(S)).
for(uint64_t sub=S;sub;sub&=sub-1){ int i=__builtin_ctzll(sub);
}
// All submasks, including zero: O(2^popcount(S)).
for(int sub=S;;sub=(sub-1)&S){
// Process sub.
 if(!sub)break; }
// Original KACTL alternative: all submasks except S,
// including zero. O(2^popcount(S)).
for(int x=S;x;){--x&=S;
// Process x.
}
// All masks and all their submasks together: O(3^n).
// Supersets inside n bits: O(2^(n-popcount(S))).
// 0 <= n < 64; S must fit inside the lowest n bits.
uint64_t full=(1ULL<<n)-1; for(uint64_t sub=S;;sub=(sub+1)|S){
// Process sub.
 if(sub==full)break; }
// Gosper: next mask with the same number of set bits.
// Original KACTL expression, using unsigned arithmetic.
// x != 0; stop if r==0 or r is outside the n-bit range.
uint64_t c=x&-x, r=x+c; uint64_t next=(((r^x)>>2)/c)|r;
// Fixed-popcount enumeration: 0 <= k <= n < 64.
// Handle k==0 separately: only mask zero.
for(uint64_t sub=(1ULL<<k)-1;sub<(1ULL<<n);){
// Process sub; this loop requires k > 0.
 uint64_t c=sub&-sub, r=sub+c; if(r>=(1ULL<<n))break;
 sub=(((r^sub)>>2)/c)|r; }
// SOS DP: original KACTL subset transform, array D[1<<K].
// D[S] becomes the sum of original D[sub] for sub subset S.
rep(b, 0, K)rep(i, 0, (1<<K)) if(i&1<<b)D[i]+=D[i^(1<<b)];
// Superset transform: sum over all supersets of S.
rep(b, 0, K)rep(i, 0, (1<<K)) if(!(i&1<<b))D[i]+=D[i|(1<<b)];
// Inverse subset / superset transforms (Mobius).
rep(b, 0, K)rep(i, 0, (1<<K)) if(i&1<<b)D[i]-=D[i^(1<<b)];
rep(b, 0, K)rep(i, 0, (1<<K)) if(!(i&1<<b))D[i]-=D[i|(1<<b)];
// Each transform: O(K*2^K) time, O(2^K) space.
// Bitset 0/1 subset sum: each weight is used at most once.
bitset<MAXN>dp;dp[0]=1; for(int W:weights)dp|=dp<<W;
// About O(items*MAXN/word_bits); sums >= MAXN are dropped.
// See the bitset snippet for masks and restricted transitions.
// Power of two; exclude zero.
bool power_of_two=S&&!(S&(S-1));
// Remove the lowest set bit.
S&=S-1;
// Gray code: consecutive codes differ in one bit.
uint64_t gray=S^(S>>1);
```

11.3 Math Reference

11.3.1 Euler's totient

For $n \geq 1$, $\phi(n)$ counts $1 \leq a \leq n$ with $\gcd(a, n) = 1$. For prime p, $k \geq 1$, distinct prime divisors p_i, and $\gcd(m, n) = 1$:

$$\phi(1) = 1, \qquad \phi(p^k) = (p-1)p^{k-1},$$

$$\phi(mn) = \phi(m)\phi(n), \qquad \phi(n) = n \prod_{p_i | n} \left(1 - \frac{1}{p_i}\right).$$

11.3.2 Dirichlet convolution and Möbius

Let μ be the Möbius function and $[P]$ be 1 when P holds, else 0. For $n \geq 1$, write $\mathbf{1}(n) = 1$, $\mathrm{id}(n) = n$, $\varepsilon(n) = [n = 1]$, and define

$$(f * g)(n) = \sum_{d|n} f(d)g(n/d).$$

$$\sum_{d|n} \phi(d) = n, \qquad \phi * \mathbf{1} = \mathrm{id},$$

$$\sum_{d|n} \mu(d) = [n = 1], \quad \mu * \mathbf{1} = \varepsilon,$$

$$\sum_{d|n} \mu(d)\frac{n}{d} = \phi(n), \qquad \mu * \mathrm{id} = \phi.$$

For positive i, j:

$$[\gcd(i, j) = 1] = \sum_{d|\gcd(i,j)} \mu(d).$$

11.3.3 Fermat and Euler

For prime p, $a^p \equiv a \pmod p$ for every integer a; if $p \nmid a$, $a^{p-1} \equiv 1 \pmod p$. For $\gcd(a, m) = 1$, $a^{\phi(m)} \equiv 1 \pmod m$. For $a \geq 0$, $m \geq 2$, $b \geq 0$, put $t = \phi(m)$:

$$a^b \equiv \begin{cases} a^{b \bmod t}, & \gcd(a, m) = 1, \\ a^b, & \gcd(a, m) > 1,\ b < t, \\ a^{b \bmod t + t}, & \gcd(a, m) > 1,\ b \geq t \end{cases} \pmod m.$$

11.3.4 Wilson

For $n > 1$, n is prime iff $(n-1)! \equiv -1 \pmod n$. For composite $n > 4$, $(n-1)! \equiv 0 \pmod n$.

11.3.5 Sprague–Grundy

For a finite impartial game under normal play (no moves loses), with no cycles, let $\mathrm{next}(x)$ be the possible next states. The mex is the smallest nonnegative integer absent from a set.

$$SG(x) = \mathrm{mex}\{SG(y) : y \in \mathrm{next}(x)\}.$$

$SG(x) = 0$ iff x is losing. For independent games:

$$SG(G_1 + \cdots + G_k) = SG(G_1) \oplus \cdots \oplus SG(G_k).$$

11.3.6 Floor-division blocks

For $1 \leq l \leq n$, set $q = \lfloor n/l \rfloor$ and $r = \lfloor n/q \rfloor$. Then $\lfloor n/i \rfloor = q$ for all $i \in [l, r]$. Advance $l = r + 1$; there are at most $2\lfloor\sqrt{n}\rfloor$ blocks. With $F(x) = \sum_{i=1}^{x} f(i)$:

$$\sum_{i=1}^{n} f(i)g(\lfloor n/i \rfloor) = \sum_{\text{blocks } [l,r]} (F(r) - F(l-1))g(q).$$

11.3.7 Reordering sums

For $h = \min(n, m)$ and positive n, m:

$$\begin{aligned} &\sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{d|\gcd(i,j)} f(d) \\ &\quad = \sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{d=1}^{h} [d \mid i][d \mid j] f(d) \\ &\quad = \sum_{d=1}^{h} f(d) \left\lfloor \frac{n}{d} \right\rfloor \left\lfloor \frac{m}{d} \right\rfloor. \end{aligned}$$

For $k \geq 1$, put $N = \lfloor n/k \rfloor$, $M = \lfloor m/k \rfloor$, and $H = \min(N, M)$. Counting pairs with fixed gcd:

$$\begin{aligned} &\sum_{i=1}^{n}\sum_{j=1}^{m} [\gcd(i, j) = k] \\ &\quad = \sum_{a=1}^{N}\sum_{b=1}^{M} [\gcd(ka, kb) = k] \\ &\quad = \sum_{a=1}^{N}\sum_{b=1}^{M} [\gcd(a, b) = 1] \\ &\quad = \sum_{d=1}^{H} \mu(d) \left\lfloor \frac{n}{kd} \right\rfloor \left\lfloor \frac{m}{kd} \right\rfloor. \end{aligned}$$

$$\sum_{i=1}^{n}\sum_{j=1}^{m} A_i B_j = \left(\sum_{i=1}^{n} A_i\right)\left(\sum_{j=1}^{m} B_j\right).$$

11.3.8 Pick's theorem

For a simple polygon with integer-coordinate vertices, let I be the number of interior lattice points and B the number on its boundary.

$$A = I + \frac{B}{2} - 1, \qquad B = \sum_{\text{edges } (dx,dy)} \gcd(|dx|, |dy|).$$

11.4 Small Utilities

SmallUtilities.cpp (14 lines)

```
// Coordinate compression: sorted distinct values, 0-based IDs.
vec<ll>values={7, 3, 7}, v=values;sort(all(v));
v.erase(unique(all(v)), v.end());
auto get_id=[&](ll x){
 return lower_bound(all(v), x)-v.begin(); };
cout<<get_id(7); // 1
// Precompute floor(log2(i)); query only i>=1.
int n=8;vi lg(n+1); rep(i, 2, n+1)lg[i]=lg[i/2]+1;
cout<<lg[8]; // 3
// Read a whole line after >>; keep leading spaces.
int x;cin>>x;
cin.ignore(numeric_limits<streamsize>::max(), '\n');
string line;getline(cin, line);cout<<line;
// stdin: 7\n  text\n -> line is "  text"
```

Techniques (A)

techniques.txt

Usage:

```
# For offline connectivity with undo, use RollbackUF:
RollbackUF uf(3);int t=uf.time();uf.join(0,1);
cout<<uf.size(0); // 2
uf.rollback(t);cout<<uf.size(0); // 1
```

159 lines

```
Recursion
Divide and conquer
  Finding interesting points in N log N
Algorithm analysis
  Master theorem
  Amortized time complexity
Greedy algorithm
  Scheduling
  Max contiguous subvector sum
  Invariants
  Huffman encoding
Graph theory
  Dynamic graphs (extra book-keeping)
  Breadth first search
  Depth first search
  * Normal trees / DFS trees
  Dijkstra's algorithm
  MST: Prim's algorithm
  Bellman-Ford
  Konig's theorem and vertex cover
  Min-cost max flow
  Lovasz toggle
  Matrix tree theorem
  Maximal matching, general graphs
  Hopcroft-Karp
  Hall's marriage theorem
  Graphical sequences
  Floyd-Warshall
  Euler cycles
  Flow networks
  * Augmenting paths
  * Edmonds-Karp
  Bipartite matching
  Min. path cover
  Topological sorting
  Strongly connected components
  2-SAT
  Cut vertices, cut-edges and biconnected components
  Edge coloring
  * Trees
  Vertex coloring
  * Bipartite graphs (=> trees)
  * 3^n (special case of set cover)
  Diameter and centroid
  K'th shortest path
  Shortest cycle
Dynamic programming
  Knapsack
  Coin change
  Longest common subsequence
  Longest increasing subsequence
  Number of paths in a dag
  Shortest path in a dag
  Dynprog over intervals
  Dynprog over subsets
  Dynprog over probabilities
  Dynprog over trees
  3^n set cover
  Divide and conquer
  Knuth optimization
  Convex hull optimizations
  RMQ (sparse table a.k.a 2^k-jumps)
  Bitonic cycle
  Log partitioning (loop over most restricted)
Combinatorics
  Computation of binomial coefficients
  Pigeon-hole principle
  Inclusion/exclusion
  Catalan number
  Pick's theorem
Number theory
  Integer parts
  Divisibility
  Euclidean algorithm
  Modular arithmetic
  * Modular multiplication
  * Modular inverses
  * Modular exponentiation by squaring
  Chinese remainder theorem
  Fermat's little theorem
  Euler's theorem
  Phi function
  Frobenius number
  Quadratic reciprocity
  Pollard-Rho
  Miller-Rabin
  Hensel lifting
  Vieta root jumping
Game theory
  Combinatorial games
  Game trees
  Mini-max
  Nim
  Games on graphs
  Games on graphs with loops
  Grundy numbers
  Bipartite games without repetition
  General games without repetition
  Alpha-beta pruning
Probability theory
Optimization
  Binary search
  Ternary search
  Unimodality and convex functions
  Binary search on derivative
Numerical methods
  Numeric integration
  Newton's method
  Root-finding with binary/ternary search
  Golden section search
Matrices
  Gaussian elimination
  Exponentiation by squaring
Sorting
  Radix sort
Geometry
  Coordinates and vectors
  * Cross product
  * Scalar product
  Convex hull
  Polygon cut
  Closest pair
  Coordinate-compression
  Quadtrees
  KD-trees
  All segment-segment intersection
Sweeping
  Discretization (convert to events and sweep)
  Angle sweeping
  Line sweeping
  Discrete second derivatives
Strings
  Longest common substring
  Palindrome subsequences
  Knuth-Morris-Pratt
  Tries
  Rolling polynomial hashes
  Suffix array
  Suffix tree
  Aho-Corasick
  Manacher's algorithm
  Letter position lists
Combinatorial search
  Meet in the middle
  Brute-force with pruning
  Best-first (A*)
  Bidirectional search
  Iterative deepening DFS / A*
Data structures
  LCA (2^k-jumps in trees in general)
  Pull/push-technique on trees
  Heavy-light decomposition
  Centroid decomposition
  Lazy propagation
  Self-balancing trees
  Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)
  Monotone queues / monotone stacks / sliding queues
  Sliding queue using 2 stacks
  Persistent segment tree
```